MICROCONTROLADORES 2

Bitácora de diseño

Cambios, problemas y soluciones · Robot de laberinto

EQUIPO

Manuel Harari López

Víctor Miguel Vázquez

Carlos Emiliano Escamilla Garibay

FECHA

12 de noviembre de 2026

VERSIÓN

Revisión B7 · documento 2.0

Contenido

Cómo leer esta bitácora

Línea de tiempo

Etapas

Etapa 1. Primera versión: UNO Q con encoders (19-oct)
Etapa 2. Movimiento por tiempo y dos iteraciones (19-oct)
Etapa 3. Bus I2C, batería y primeros ajustes de chasis (25-oct)
Etapa 4. Firmware C++ bare-metal y rediseño del chasis a 120×120 (28-oct)
Etapa 5. Chasis modular R3 / UNO Q y documentación de apoyo (29-oct)
Etapa 6. Ajustes de software del UNO Q (29-oct)
Etapa 7. Herramientas del repositorio (1-nov)
Etapa 8. Medidas reales de rueda, motor y pila (7-nov)
Etapa 9. Soporte del VL53L0X: de postes con ranura a una pieza aparte con insertos (7-nov)
Etapa 10. Piso 2 modelado igual al ya impreso (7-nov)
Etapa 11. Cuna del motor: ajuste fino del hueco (7-nov)
Etapa 12. OLED en la cabeza de la cámara y mástil 2020 (7-nov)
Etapa 13. Diagrama de conexiones (7-nov)
Etapa 14. Cámara en el UNO Q: búsqueda del nodo de video (7 y 8-nov)
Etapa 15. Diagnóstico en la OLED (8-nov)
Etapa 16. Panel web de calibración de colores (8-nov)
Etapa 17. Manuales (8-nov)
Etapa 18. Pedestal impreso y cabeza de cámara (8-nov)
Etapa 19. Panel de cámara, cabeza en el CAD y motores invertidos (8 al 10-nov)
Etapa 20. La cámara como sensor de paredes con mapa 16×16 (10-nov)
Etapa 21. Laberinto 11×11, modo “tonto” y “los ToF mandan” (10-nov)
Etapa 22. Odometría ligera y confianza en el ToF frontal (10-nov)
Etapa 23. Frenado, `D_FRENTE` y simulador “qué haría” (10-nov)
Etapa 24. Solo seguir pared, prueba sin cámara y escritura atómica (10-nov)
Etapa 25. Explorar hasta el amarillo; la cámara guía y cubre puntos ciegos (10-nov)
Etapa 26. Robustez de la cámara y la web, colores de fábrica y manuales (11-nov, madrugada)
Etapa 27. Alinear con giros de 5 grados y su retirada (11-nov)
Etapa 28. Reescritura: avance por distancia, cámara solo para amarillo y mano izquierda (11-nov)
Etapa 29. Mapa de paredes y colores con ruta corta; cambios bruscos de ToF; cámara blanco/negro (11-nov)
Etapa 30. Máquina de estados, votos, frente libre a 220 mm, alto en amarillo desde el STM32 y web sin Bridge (11-nov)
Etapa 31. MPU-6050 opcional: giros por ángulo y rumbo sostenido (11-nov)
Etapa 32. Calibración de los sensores de distancia desde la página web (11-nov)
Etapa 33. Dos versiones del sketch: con y sin MPU-6050 (11-nov)

Etapa 34. Cámara monocromática OV9281: solo blanco y negro (11-nov)
Etapa 35. Exprimir la cámara monocromática: rumbo, umbral automático y 2 técnicas experimentales (11-nov)
Etapa 36. Umbrales del frontal en milímetros reales (11-nov)
Etapa 37. Mano izquierda pura, compensación de motores y alto a 10 mm (11-nov)
Etapa 38. Exploración completa: la meta al centro (11-nov)
Etapa 39. Primera corrida larga con MPU: 3 fallas del log (11-nov)
Etapa 40. Cronómetro, registro de colores y alineación con MPU (11-nov)
Etapa 41. El MPU completo: choques, atorones, giros verificados y robot levantado (11-nov)
Etapa 42. Un solo sketch, con MPU-6050 (11-nov)
Etapa 43. Un solo Python (11-nov)
Etapa 44. Centrado en el carril con la geometría del chasis; choques (11-nov)
Etapa 45. Choques confirmados (11-nov)
Etapa 46. Pasillos de un tirón; golpe a 1.5 g (11-nov)
Etapa 47. Exploración hacia el centro, sin cámara (12-nov)
Etapa 48. Simulación 2D: reglas de ToF y MPU (12-nov)

Pruebas realizadas

Pruebas en pista
Mediciones e iteraciones físicas
Electrónica y alimentación
Sensores
Software
Herramientas de prueba integradas en el robot
Pendiente

Decisiones de ingeniería

Arquitectura según el grafo de conocimiento

Lecciones aprendidas

Pendientes

Glosario

Cómo leer esta bitácora

EN CORTO

La bitácora cuenta cómo se construyó el robot del 19 de octubre al 11 de noviembre de 2026: 46 etapas, de la primera versión con encoders a la exploración completa con MPU-6050. Hubo dos iteraciones sobre el mismo chasis: A (ATmega328p, respaldo) y B (Arduino UNO Q, la actual).

Este documento cuenta cómo se construyó el robot de laberinto de Microcontroladores 2, del 19 de octubre al 11 de noviembre de 2026, a partir del historial de commits del repositorio, de los análisis de la última sesión de trabajo (11-nov), que no quedaron en commits, y de las respuestas del equipo sobre sus pruebas en pista. El robot recorre un laberinto de celdas de 18 cm, encuentra una marca amarilla (la meta), cuenta marcas de color en el piso y en una segunda corrida debe ir por el camino más corto que conoce.

Hubo dos iteraciones sobre un mismo chasis impreso en 3D, de modo que el chasis acepta ambas placas:

- **Iteración A: ATmega328p (Arduino Nano / UNO R3).** Todo el control corre en el microcontrolador, programado en C++ sin librerías de Arduino, con tres sensores de distancia VL53L0X (ToF) y un sensor de color TCS34725. No tiene cámara. Es el respaldo.
- **Iteración B: Arduino UNO Q (versión actual).** Un STM32 con control de tiempo real y un Linux con Python y una cámara USB. Es la versión con la que se sigue trabajando.

Cómo leer la bitácora. La sección “Línea de tiempo” resume todo en una tabla. “Etapas” detalla cada etapa con el mismo formato: qué cambió, el problema, la solución, las pruebas o evidencia, el razonamiento, el resultado y los commits. El equipo probó el robot en pista durante todo el desarrollo (laberinto de 2 m x 2 m con paredes de madera); sus respuestas del 11-nov están en “Pruebas en pista”. Lo que falta es probar en pista la última versión (distancia, saltos, votos, máquina de estados, MPU) y se repite en pendientes.

⚠ CUIDADO

Una regla de honestidad atraviesa todo el documento: cuando un motivo o una prueba no está escrito en el commit, el diff, el código ni en las respuestas del equipo, se marca **(deducido)**. Los valores de calibración que aparecen en las primeras etapas son valores iniciales de partida, no resultados medidos, salvo donde se dice lo contrario.

Línea de tiempo

EN CORTO

Una fila por etapa: fecha, cambio clave y por qué. La figura agrupa las 46 etapas en cinco fases.

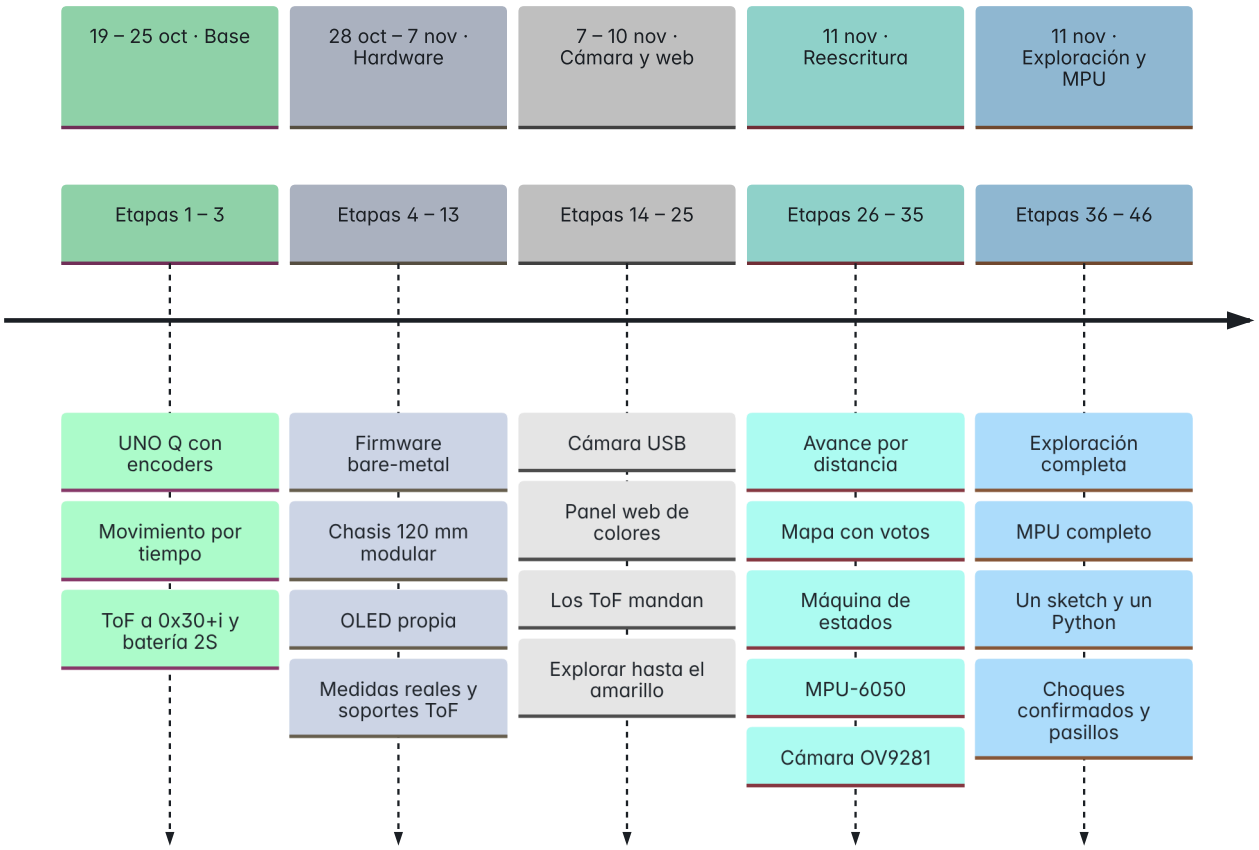


Figura 1. Etapas del proyecto. Qué mirar: casi la mitad de las etapas ocurre el 11-nov, cuando el robot pasa de reglas por tiempo a medir con los ToF y el MPU.

Fecha	Etapa	Cambio clave	Por qué
19-oct	1	Primera versión UNO Q con encoders, flood-fill y borrado de mapa con botón	Base del software de laberinto por corridas
19-oct	2	Movimiento por tiempo; se divide el repo en `uno_q/`, `atmega328p/` y `chasis/`	Los motores N20 no traen encoder
25-oct	3	ToF con dirección 0x30+i antes de iniciar; batería 2S; modo calibración; R3 como única versión (por un tiempo)	El TCS34725 y un VL53L0X comparten 0x29
28-oct	4	Firmware C++ bare-metal (42% de flash); chasis de 120 mm y 2 pisos	RAM al 72% con Arduino y límite de 120 mm con ruedas

Fecha	Etapa	Cambio clave	Por qué
29-oct	5	Chasis modular nano / unoq / uno_r3; mismo mapa de pines; `docs/uno_q.md`	Un solo chasis para ambas placas
29-oct	6	Driver OLED propio para el UNO Q	Adafruit_SSD1306 no compila en Zephyr
1-nov	7	graphify y skills de ponytail	Menos contexto al explorar el código
7-nov	8	Medidas reales de rueda (45×17), motor y pila (70 mm)	El CAD tenía medidas estimadas
7-nov	9	Soporte ToF: de postes con ranura de 1.8 mm a pieza aparte tipo bloque con insertos	El sensor "bailaba" y no se podía roscar en los laterales
7-nov	10	Piso 2 modelado igual al ya impreso	No reimprimir
7-nov	11	Hueco del motor: 11.94, 12, 12.05, 11.85, 11.9 mm	Encontrar el ajuste a presión
7-nov	12	OLED a la cabeza de la cámara; espera de 500 ms al arrancar	Una pieza menos; la OLED no encendía por VIN
7-nov	13	Diagrama de conexiones generado por script	Un diagrama para R3 y Q
7 y 8-nov	14	La cámara prueba todos los `/dev/video*` y usa el primero que entrega cuadro	Una cámara USB expone varios nodos
8-nov	15	Contador `PY n` en la OLED	Saber si Python corre sin consola
8-nov	16	Panel web para calibrar colores	No copiar valores a mano
8-nov	17	Manuales de uso y técnico, en Markdown y PDF	Operar y explicar el robot
8-nov	18	Pedestal impreso M4 con inserto; cabeza de cámara con tolerancias	No hay perfil 2020 a mano
8 al 10-nov	19	Panel de cámara en vivo; motores invertidos por software	Ajustar regiones a ojo
10-nov	20	La cámara detecta paredes hasta 2 celdas adelante (mapa 16×16)	Anticipar para no entrar a callejones
10-nov	21	Mapa 11×11; "los ToF mandan" (la cámara solo encarece)	La cámara podía bloquear rutas por error
10-nov	22	Odometría ligera y frontal con tope de celdas; `PARED_F = 130`	El tiempo deriva
10-nov	23	`D_FRENTE` 55 a 130 mm; frenado y simulador "qué haría"	El sensor "nunca llega a 50"
10-nov	24	Solo seguir pared; escritura atómica de datos	Estrategia mínima; JSON roto tumbaba la app
10-nov	25	Meta: explorar hasta el amarillo; cámara desempata	La meta real es una marca amarilla
11-nov	26	Cámara con reconexión en hilo; colores de fábrica	El hub tarda en arrancar

Fecha	Etapa	Cambio clave	Por qué
11-nov	27	Alinear con giros de 5 grados y su retirada	Giros por tiempo dejan el rumbo chueco
11-nov	28	Reescritura: avance por distancia, cámara solo amarillo, mano izquierda	El tiempo deriva; los ToF ya decidían todo
11-nov	29	Mapa de paredes y colores con ruta corta; cambios bruscos de ToF; blanco/negro	Que la segunda corrida no explore de nuevo
11-nov	30	Máquina de estados, votos, frente libre a 220 mm, alto en amarillo desde el STM32, web sin Bridge	Estabilidad y latencia
11-nov	31	MPU-6050 opcional autodetectado	Error de rumbo por giros de tiempo
11-nov	32	Calibración de los ToF desde la web	Error de ±15 mm frente a umbrales
11-nov	33	Dos versiones del sketch: con y sin MPU-6050	Poder correr sin el MPU
11-nov	34	Cámara monocromática OV9281: solo blanco y negro	La cámara real no ve colores
11-nov	35	Cámara: exposición fija, Otsu, rumbo por punto de fuga; experimental: distancia y aberturas	Desviación en recta y esquinas sin ver

Etapas

EN CORTO

Cada etapa sigue el mismo formato: qué cambió, el problema, la solución, las pruebas o evidencia, el razonamiento, el resultado y los commits. Las últimas etapas (36 a 46) vienen de la sesión del 11-nov y de las respuestas del equipo.

Etapa 1. Primera versión: UNO Q con encoders (19-oct)

Qué cambió.	Se crea `laberinto/` con un sketch para el STM32 (L298N, encoders, TCS34725, OLED, botón) y una app Python con cámara y flood-fill (283 líneas en `main.py`, 332 en `sketch.ino`). Después, una pulsación larga del botón (más de 2 s) borra la memoria (mapa en `/app/mapa.json` y conteo de colores) y la OLED muestra LISTO al volver al inicio en la primera corrida.
Problema.	El laberinto cambia entre pruebas. Sin forma de olvidar el mapa guardado, la corrida 2 usaría datos viejos.
Solución.	El sketch devuelve `arranque = 2` si el botón se mantuvo más de 2000 ms; Python llama `borrar()`, que reinicia paredes, visitas, marcas y corrida, y borra el archivo.
Pruebas / evidencia.	Ninguna en los commits; el README solo trae una lista de calibración por hacer (colores, avance, KP, cámara).
Razonamiento.	Diseño por corridas: la 1 explora con flood-fill; la 2 y la 3 siguen la ruta más corta con rectas rápidas. Una marca de color cuenta solo si se sostiene 3 lecturas y antes se vio piso; Python guarda la celda para no contarla dos veces.
Resultado.	Base funcional del software de laberinto, todavía con encoders en D2/D3 (`TICKS_CELDA` 360 por 18 cm, `TICKS_90` 120).
Commits.	43671b9, a257e7a, bf223fb

Etapa 2. Movimiento por tiempo y dos iteraciones (19-oct)

Qué cambió.	`USE_ENC` pasa a 0 (movimiento por tiempo): `MS_CELDA` 600 ms por 18 cm, `MS_RAPIDA` 400 ms, `MS_90` 250 ms; se agrega `AJUSTE_DER` (suma PWM al motor derecho) y `OFF_COLOR` pasa a celdas (0.3). Luego el repo se divide en `uno_q/` (3 VL53L0X para centrar y frenar; la cámara adelanta paredes de la celda siguiente), `atmega328p/` (todo en el MCU, flood-fill, EEPROM, Timer1 de 10 ms, 517 líneas) y `chasis/` (OpenSCAD paramétrico y planos PDF).
Problema.	Los motores N20 no tienen encoders (el commit y el README posterior lo dan como hecho).

Solución.	Avance y giro por tiempo con rampa de velocidad (`VEL_MIN` 80, `VEL_EXPLORA` 140, `VEL_RAPIDA` 220, `VEL_GIRO` 130). En lugar de encoders, los VL53L0X corrigen centrado y frenado. En el ATmega los motores usan PWM D5/D6 (Timer0) porque Timer1 genera la base de 10 ms.
Pruebas / evidencia.	No hay mediciones; los tiempos son valores iniciales por calibrar.
Razonamiento.	Dos iteraciones sobre el mismo chasis: A (ATmega, sin cámara) y B (UNO Q con visión). Se elimina la dependencia de encoders cambiando precisión por simplicidad mecánica, y se compensa con sensores de distancia.
Resultado.	Dos bases de firmware y un chasis común.
Commits.	2edde2e, e8b9b4c

Etapa 3. Bus I2C, batería y primeros ajustes de chasis (25-oct)

Qué cambió.	Los VL53L0X cambian su dirección a 0x30+i antes de `init()` y el TCS34725 se inicializa al final. Batería 2S 18650 con interruptor a VIN. Diagrama de conexiones del UNO R3. Piso 3 para mini protoboard, OLED y botón. Luego tolerancias FDM, STL, manuales y BOM, y un modo calibración con botón (corto avanza una celda, largo de más de 1 s gira 90 grados).
Problema.	El TCS34725 también responde en 0x29, igual que un VL53L0X sin reasignar; con ambos en el bus la dirección chocaba (lo dice el mensaje del commit).
Solución.	Cambiar la dirección de los ToF antes de `init()` y reordenar el arranque (ToF primero, color después). El TCS queda con LED/INT sin conectar (LED siempre encendido).
Pruebas / evidencia.	Se imprime una pieza de prueba (`PIEZA="prueba"`) para ajustar `HOLGURA` antes de imprimir todo: 0.2 mm por lado, M3 pasante 3.4, M3 a rosca 2.8, M2 a rosca 1.8, botón 7 mm (D = 7.3), switch 6 mm. Uso de memoria del sketch Arduino en el ATmega: 18.9 KB de flash (58%) y 1.47 KB de RAM (72%), según el README. No hay evidencia de que la pieza de prueba se imprimiera en este tramo.
Razonamiento.	Desde 25ce73b el README declara el R3 como única versión y deja `uno_q/` como “descartada, sin mantenimiento”; los planos quedan solo para R3. Esa decisión se revierte en la etapa 5, cuando el UNO Q vuelve como placa principal. Se separa la lectura de color y la de distancia en el arranque para evitar el choque de direcciones sin hardware extra.
Resultado.	Chasis de 3 pisos para R3, STL listos, manuales y BOM.
Commits.	883e2a1, 5d436bc, a65d3ec, 25ce73b

Etapa 4. Firmware C++ bare-metal y rediseño del chasis a 120×120 (28-oct)

Qué cambió.	El sketch se reemplaza por C++ sin librerías de Arduino, para Microchip Studio, con acceso a registros: TWI a 400 kHz, Timer1 en CTC a 1 kHz, Timer0 Fast PWM en OC0A/OC0B, SSD1306, TCS34725 y VL53L0X (librería de Pololu adaptada). Parámetros: `T_CELDA` 60 ticks (10 ms), `T_RAPIDA` 40, `T_90` 25, `PARED` 120 mm, `D_FRENTE` 34, `D_LADO` 38, `KC` 0.8. Chasis: de 125 a 120 mm, 2 pisos, sensores que deslizan sin tornillos; luego se modelan pines y conectores, se reubican OLED, botón e interruptor, se rediseña para headers rectos (batería sobre el eje, `BAT_Y` 7; pasillos libres de 27 mm para el dupont de cada VL53L0X; avellanados en cunas), se modelan los cables con rutas, tolerancias A2L, placa para la cama de la X2D y manual paso a paso, y se simplifica a solo tornillos Allen M3 e insertos de calor.
Problema.	Reducir recursos (RAM al 72%), cumplir el tamaño máximo con ruedas (L = 120 mm) y resolver varios choques geométricos entre conectores y piezas, además de que la batería y los cables de VL53L0X no estorbaran.
Solución.	Bare-metal con Microchip Studio, por la materia. Geometría paramétrica. Batería acostada (65×37×19) sobre el eje. Los topes de batería de 8 mm se mueven fuera de los pasillos de conectores. Se eliminan separadores y tuercas a favor de columnas impresas en el piso 1 e insertos de calor (menos piezas, BOM más corto).
Pruebas / evidencia.	Verificaciones geométricas en OpenSCAD (intersecciones de cables en el commit posterior) y medidas de memoria: 13.8 KB de flash (42%) y 1.0 KB de RAM (50%), frente a 58% y 72% del sketch Arduino. No hay impresiones reales documentadas en este tramo.
Razonamiento.	Primero se prefirió ranura de PCB (1.6 mm + 0.4) y sensores que deslizan, sin tornillos, para facilitar el armado; luego se cambió a tornillos M3 con insertos para dar rigidez. Se descartó el piso 3 y los separadores M3 largos. La batería se movió al eje para liberar pasillos de cable, a costa de colocar la OLED sobre el hueco trasero.
Resultado.	Firmware de 42% de flash y chasis de 2 pisos con manual de ensamble.
Commits.	cdf37ec, eb0672b, 4a0368c, 8d67661, c0b858a, c32c131

Etapa 5. Chasis modular R3 / UNO Q y documentación de apoyo (29-oct)

Qué cambió.	Variantes `PLACA = nano | unoq | uno_r3` en el CAD. UNO Q con cámara USB UVC de 32×32 en un mástil, repisa con hub USB (95×30×12) y OLED en el mástil. Proto Shield con mini protoboard de 170 puntos (45.7×34.5×9.5) para el bus I2C (4 columnas: VCC, GND, SDA, SCL; VCC a 3V3 con la Q y a 5V con el R3). Columnas traseras en (±48, -40) para librar el jack y el USB-B del R3. README y

	Makefile pasan a UNO R3 (avrdude `m328p`, `/dev/ttyACM0`). El UNO Q adopta el mismo mapa de pines (ENA D5, ENB D6, IN1 a IN4 en D7 a D10, botón D2; antes D9/D10, D4/D5/D7/D8, D12). Se escribe `docs/uno_q.md` (plan, Bridge, cámara, color, energía), la guía de programación y el manual modular (pasos 7 a 11), y se regeneran manuales y planos.
Problema.	El R3 funciona como respaldo y el Q es experimental; al cambiar de placa habría que recablear. Además el R3 es de 5 V y el Q de 3.3 V: a 5 V se dañan los pines I2C del Q (aviso del README).
Solución.	Shield con dupont que se saca y se clava en la otra placa, mismo mapa de pines y un jumper de VCC 5V/3V3. Aviso de alimentación: VIN necesita al menos 7 V, así que se recarga la batería a 7.2 V.
Pruebas / evidencia.	Rutas de 37 cables verificadas por intersección en OpenSCAD. Parámetros de cámara en `uno_q.md`: lente a 150 mm, 25 grados hacia abajo, unos 100×75 grados de campo, ve el piso desde unos 3 cm; el documento dice que la altura de la pared y el tamaño del mástil estaban pendientes (el hub: “medir el suyo”). Plazo declarado: competencia en 15 días desde el 29-oct.
Razonamiento.	Cámara USB UVC y no MIPI-CSI, porque la CSI exige la Media Carrier (más grande, 1.8 V). El TCS34725 sigue siendo necesario porque la cámara no reemplaza una lectura de color exacta: ella anticipa y el sensor confirma. R3 de respaldo oficial, Q como placa principal experimental. El chasis común evita imprimir dos.
Resultado.	Un solo chasis para ambas placas, con la Q como extra (mástil, repisa, cabeza de cámara).
Commits.	43d9b37, 70bc992, 3e551e3, 76a9c97, 72eac0e, 36310ab, 2ca6fee, df9381c, 188c5ee, 21e7d1e

Etapa 6. Ajustes de software del UNO Q (29-oct)

Qué cambió.	Driver SSD1306 propio (`uno_q/sketch/oled.h`, sin librerías), OpenCV fijado en `requirements.txt` y luego quitado, mensaje de arranque en Python.
Problema.	Adafruit_SSD1306 no compila en el core Zephyr del UNO Q (mensaje del commit). OpenCV ya viene en la imagen base, así que el requirement sobraba.
Solución.	Driver propio y `requirements.txt` sin OpenCV.
Pruebas / evidencia.	Compilaciones fallidas implícitas, sin detalle. El cambio de versión de OpenCV y su retirada aparecen en dos commits seguidos, lo que sugiere prueba y error en la placa (deducido).
Razonamiento.	Evitar dependencias externas que no compilan en la Q.
Resultado.	Sketch de la Q compilable.
Commits.	7a777fc, 0b2ebf7, dfcc08c, b82d8a3

Etapa 7. Herramientas del repositorio (1-nov)

	Etapa menor. Se instalan graphify (grafo del código con hooks) y los skills de ponytail, y `CLAUDE.md` pide consultar el grafo antes de buscar código, para reducir contexto al explorar. No hay problema de producto ni pruebas. Resultado: repositorio con grafo de código versionado.
Commits.	44adb1e, fc9c6cb, b170e86

Etapa 8. Medidas reales de rueda, motor y pila (7-nov)

Qué cambió.	Se miden las piezas reales y se actualiza el CAD: rueda D 45 × 17 (antes 34 × 20), motor 26.11 × 11.94 × 11.94 (antes 24×12×10), pila 70 mm (antes 65). `POZO_Y` se calcula como media cuerda de la rueda más 1.5 mm. Postes delanteros en (±34, 55) (antes 42, 50) y `TOF_LADO` en [43, 40] (antes 50, 34.5) para librar la rueda grande. Se regeneran planos, manuales, BOM, cables y renders con etiquetas de progreso.
Problema.	La rueda real era más grande que la modelada y chocaba con sensores y columnas.
Solución.	Actualizar las dimensiones a las reales y reubicar sensores y columnas.
Pruebas / evidencia.	Medidas físicas de rueda, motor y pila.
Razonamiento.	Modelar con la pieza real y no con el catálogo; el límite de 120 mm con ruedas obliga a que los sensores se acomoden a la rueda y no al revés.
Resultado.	CAD y documentación alineados con las piezas reales.
Commits.	cef8925, 5c5f2ed, f8137e9, b33987a, ef31a48, d22d141, 176e066 (ver también la etapa 9)

Etapa 9. Soporte del VL53L0X: de postes con ranura a una pieza aparte con insertos (7-nov)

Qué cambió.	El soporte pasó por tres formas el mismo día. - Forma 1: ranura de 1.8 mm (`RAN_TOF` , antes 2.0) con postes que abrazan 4 mm de cada oreja del sensor; luego un tornillo por oreja. M2×8 pasa a M3×8 (rosca de 2.5 mm en plástico, paso de 3.4) porque la oreja real pasa M3; `TOF_AG = [20, 5.35]` confirmado con el plano de UNIT. - Forma 2: pieza impresa aparte (`soporte_tof` , se imprimen 3: frente, izquierdo, derecho) que se atornilla al piso 1 con 2 tornillos M3×8 desde abajo y 2 insertos. Aparece `TOF_BASE = 7` mm (el sensor queda 7 mm sobre el piso 1) y `TOF_FIJ` . Las columnas delanteras pasan de (34, 55) a (32, 55.5) y el cable frontal sube de 50 a 56 mm. - Forma 3, tras medir el módulo: alto de 10.7 a 11 mm, y el agujero de fijación a 5.5 mm del borde de los pines

	(`TOF_AG = [20, TOF[1] - 5.5]`), luego subido 1.5 mm. El soporte pasa a un bloque detrás de la placa con 2 insertos y tornillos M3×10 por el frente, sin ranura, de modo que el header y los dupont quedan libres arriba. Los insertos de abajo se mueven al centro del bloque (`TOF_FIJ = [8, -5.3]`). Muesca cilíndrica (d = 9, luego 10 mm, en x = ±15.5) para librar la columna delantera del piso 1 en los soportes laterales. Bloque 6 mm más ancho para dar pared a los insertos, luego reducido a +3.4 mm (2.2 mm de pared) para no chocar con el piso 1. Torres laterales hasta el tope de la placa, con el centro libre para los dupont.
Problema.	Con ranura de 2.0 mm el sensor “bailaba” (comentario del CAD). El soporte integrado obligaba a reimprimir el piso 1 cada vez que se ajustaba el sensor (deducido del diff; el mensaje no lo dice). El diseño de postes con ranura no sujetaba bien o estorbaba el header (deducido del diff). En los laterales, la columna del piso 2 tapa la cara trasera y no se podía roscar el tornillo.
Solución.	Iteración de medidas y geometría hasta un bloque detrás del módulo, con insertos, M3×10 por el frente, 2.2 mm de pared y sin choque con el piso 1.
Pruebas / evidencia.	Medición directa del módulo (11 mm de alto, agujero a 5.5 mm del borde de los pines) y plano de UNIT. La holgura del sensor suelto refleja una impresión real (inferido del comentario “con 0.4 bailaba”). El agujero se subió 1.5 mm en otro commit, lo que indica una corrección al comparar con la pieza. El repositorio no guarda el resultado de cada impresión, solo la sucesión de valores (ancho +6 mm y luego +3.4 mm; cilindro de muesca de 9 a 10 mm). M2 se descarta al verificar que el agujero de la oreja pasa M3. `prueba.stl` se regenera junto con los soportes.
Razonamiento.	Separar el soporte permite reimprimir una pieza pequeña (tres copias idénticas) en lugar del piso completo. Compromiso: la base de 7 mm sube el sensor y obliga a subir el cable frontal y a mover las columnas delanteras. Se eligen insertos con tornillo desde abajo frente a roscar en plástico (visible en el diff; el mensaje no da más motivo). El tope del bloque queda debajo del plástico del header (`zt`) para no estorbar los dupont. Compromiso entre pared de insertos (3 mm, luego 2.2 mm) y choque con el piso 1.
Resultado.	Soporte tipo bloque que se ajusta de forma independiente, sin chocar con el piso 1.
Commits.	93d9d14, 360ad39, a9cf261, 82f34ff, d328a22, 6eedd49, bd09e58, d532c3c, 7307d79, 87d42e5, 01cced4, dff5684

Etapa 10. Piso 2 modelado igual al ya impreso (7-nov)

Qué cambió.	Se modela el piso 2 para que coincida con el que ya estaba impreso: ranuras de cables de sensores en Y = 34.5 (`RAN2_Y` , antes seguían la Y de los ToF laterales) y repisa a `REPISA_Z = Z2 + ESP + 19` (postes de 19 mm, antes 82 fijo). Solo cambian los 2 agujeros delanteros, por las columnas movidas en la etapa 9.

Problema.	Para no reimprimir el piso 2, el modelo debía reflejar la pieza física y no divergir de ella.
Solución.	Se ajustan ranuras y postes de repisa en el SCAD y en `generar_pdf.py`, y se regeneran STL (piso2, placa_q, repisa) y manual.
Pruebas / evidencia.	El piso 2 ya estaba impreso; los commits dicen "igual al ya impreso" y copian sus valores. No hay medición adicional en los diffs.
Razonamiento.	Se descarta reimprimir y se adapta el modelo a la pieza física; el único cambio real aceptado son los 2 agujeros delanteros. Hubo dos commits seguidos con el mismo mensaje: el primero corrigió ranuras y postes, el segundo regeneró PDF e imágenes.
Resultado.	El piso 2 impreso sigue siendo válido salvo por los 2 agujeros delanteros.
Commits.	bd103c5, 6d85d90

Etapa 11. Cuna del motor: ajuste fino del hueco (7-nov)

Qué cambió.	Se mide el motor: 12 × 11 mm (`MOT_A = 12`, `MOT_H = 11`; antes 11.94 × 11.94). Se regeneran cuna, patín y capucha. Después el hueco se afina por pruebas de impresión: 11.94 + 0.1 por lado (12.04), 12.00, 12.05, 11.85 y 11.90 mm. Valor final: **11.9 mm** de ancho, 18 mm de largo y `MOT_H + 1` de alto.
Problema.	Ajuste de tolerancia; el motor no entraba o quedaba flojo según la impresión. Es parte de las 20 a 50 iteraciones de chasis y piezas que el equipo dice haber hecho para verificar geometrías y tolerancias.
Solución.	Pasar de una holgura calculada (`HOLGURA` global) a un valor fijo medido, porque la tolerancia de la impresora decide el ajuste.
Pruebas / evidencia.	Medición del motor y cinco commits seguidos que solo cambian el hueco, cada uno con su STL regenerado. Según el equipo fueron impresiones de prueba sucesivas, pero el repositorio no registra el resultado de cada una.
Razonamiento.	El rango probado (11.85 a 12.05) muestra la búsqueda del punto entre motor flojo y motor que no entra. Se quedó en 11.9, ligeramente menor que el motor (12), por lo que el ajuste es a presión.
Resultado.	Hueco final de 11.9 mm.
Commits.	2722f97, 712fac8, 9c03141, db48994, b5a7abb

Etapa 12. OLED en la cabeza de la cámara y mástil 2020 (7-nov)

Qué cambió.	La OLED se mueve a la cabeza de la cámara, de pie, con el header recto hacia adelante por encima de la manga; entra por arriba en 2 rieles y se elimina `clip_oled`. En el sketch, `oled.begin(..., true)` pasa a `false` (ya no va de cabeza; el último parámetro es la inversión). Mástil: `MASTIL = 20` (perfil de

	aluminio 2020, 75 mm), cabeza con hueco de 20 y opresor M3 (agujero d = 2.5), piso 2 con agujero M5 al hoyo central del mástil. El sketch espera 500 ms al arrancar por VIN.
Problema.	El clip de la OLED era una pieza extra y la pantalla iba de cabeza. La OLED no encendía al alimentar por VIN (por el comentario del código: "tarda en despertar").
Solución.	OLED integrada a la cabeza y `delay(500)` antes de iniciarla.
Pruebas / evidencia.	El `delay(500)` se añadió con el comentario "la OLED tarda en despertar al prender por VIN", lo que indica que se observó que no encendía; no hay más datos. La posición de la OLED sale de un punto calculado (`OLED_MZ = CAM_Z + 5.5 - OLED[1]`).
Razonamiento.	Se descarta el clip separado y el montaje de cabeza, que obligaba a invertir la pantalla por software: la cabeza de la cámara ya es rígida y alta, y absorbe la OLED con dos rieles. El perfil 2020 se prefiere a un mástil de 10 o 15 mm (el comentario previo decía "pendiente"), con opresor M3 a la ranura T. Esto se sustituye en la etapa 18.
Resultado.	Una pieza menos y pantalla confiable al prender.
Commits.	5a4462a, d7892a8, 9592c25

Etapa 13. Diagrama de conexiones (7-nov)

	Etapa menor de documentación. `docs/diagrama_conexiones.py` (matplotlib) genera `diagrama_conexiones.png` y `.pdf`, válido para UNO R3 y UNO Q (el cableado es el mismo), con colores de cable tomados de `cables.csv` (VCC rojo, GND negro, SDA azul, SCL turquesa, XSHUT verde, EN morado, IN café, PWR naranja, BTN rosa) para que coincida con los cables físicos. No hay problema ni pruebas; el resultado es un diagrama reproducible desde script.
Commits.	ef53524

Etapa 14. Cámara en el UNO Q: búsqueda del nodo de video (7 y 8-nov)

Qué cambió.	En `uno_q/python/main.py` : el 7-nov, en vez de abrir solo `CAM_ID` , recorre `/dev/video{CAM_ID..11}` con V4L2, usa el primero que realmente entrega un cuadro e imprime la lista de dispositivos. El 8-nov se agrega `USAR_CAM = False` para correr sin cámara, vuelve a `True` al llegar la cámara, y un print por ciclo de lo que ve (`cam: ROJO=P VERDE=. ...`).
Problema.	Una cámara USB expone varios nodos `/dev/video*` y no todos capturan, de modo que abrir uno fijo fallaba.

Solución.	Probar cada nodo con una lectura real (`cam.read()[0]`), que es más robusto que fijar un índice. El interruptor `USAR_CAM` permite probar el resto del sistema sin hardware.
Pruebas / evidencia.	El listado de `/dev/video*` y el print `cam: ...` sirven para depurar en la placa real; no hay resultados guardados.
Resultado.	Cámara activa y depurable por consola.
Commits.	98273c8, 7388d36, 3374d6a, 9784df6

Etapa 15. Diagnóstico en la OLED (8-nov)

Qué cambió.	En `sketch.ino` , un contador `nRpc` sube cada vez que Python consulta y se muestra como `PY n` junto con la última orden (`M` , `G` , `L` borrar mapa, `B` botón); la OLED se refresca cada 300 ms en reposo.
Problema.	Sin pantalla no se sabía si el programa Python en Linux estaba corriendo.
Solución.	Si `PY n` no cambia, Python no corre. Se usó la OLED, que ya existía, y no una consola serial, porque el robot corre sin PC conectada.
Pruebas / evidencia.	Ninguna medición; es instrumentación para pruebas futuras.
Resultado.	Diagnóstico visible a simple vista.
Commits.	03a7610

Etapa 16. Panel web de calibración de colores (8-nov)

Qué cambió.	Interfaz `uno_q/assets/index.html` servida en el puerto 7000 con el brick `arduino:web_ui` (`app.yaml`). `main.py` expone `/api/lectura` (r, g, b normalizados más brillo c, con el color clasificado por distancia y umbral 0.02) y `/api/guardar` (promedia 5 lecturas cada 50 ms, actualiza `REFS` , las envía al sketch con `set_ref` y las guarda en `/app/colores.json` , que se recarga al arrancar). Valores iniciales: PISO (0.33, 0.33, 0.33, 900), ROJO (0.58, 0.22, 0.20, 400), VERDE (0.26, 0.46, 0.28, 350), AZUL (0.20, 0.32, 0.48, 300), AMAR (0.42, 0.40, 0.18, 700). Después se añade un asistente paso a paso (guarda cada color en orden, con Atrás y Saltar).
Problema.	Calibrar colores obligaba a leer valores en la OLED y copiarlos a mano a `refs[]` en el sketch.
Solución.	Calibrar desde el teléfono o la PC sobre el mismo robot, con persistencia.
Pruebas / evidencia.	Cada guardado promedia 5 lecturas (50 ms entre una y otra) para reducir el ruido del sensor. El asistente se agregó después, lo que sugiere que el primer panel (botones sueltos) no bastaba (deducido). No hay datos de calibraciones reales guardados en los diffs.

Razonamiento.	Normalizar r, g, b entre sí (dividir entre su suma) hace la clasificación menos sensible a la luz. El brillo c entra con peso pequeño (0.01 sobre log) para distinguir PISO de las marcas. Si ninguna referencia queda a menos de 0.02, se devuelve PISO. Guardar en `colores.json` evita perder la calibración al reiniciar.
Resultado.	Ya no hay que tocar `refs[]` a mano.
Commits.	531b253, 3c72308

Etapa 17. Manuales (8-nov)

	Etapa menor de documentación. `docs/manual_uso.md` (uso y calibración: batería de al menos 7.4 V, unos 20 s de arranque, botón D2 con toque corto para arrancar y más de 2 s para borrar el mapa), reordenado como secuencia (sanidad, calibrar, ensayo, competencia) para poder seguirlo sin entender el sistema. Primero se escribió un manual para jueces y luego uno técnico (`manual_tecnico.md`) con explicaciones simples. Se agregan `prompt_pdf.md` (retirado después), `md_pdf.sh/.css/.js` y los PDF a color en `docs/pdf/`. Los criterios de operación (batería de al menos 7.4 V; con menos de 7 V el L298N pierde fuerza; unos 20 s de arranque) los escribió el equipo; los commits no dicen cómo se midieron.
Commits.	989c327, 7a4781a, f3b0647, c0c54fb, 7a015f0

Etapa 18. Pedestal impreso y cabeza de cámara (8-nov)

Qué cambió.	Pedestal impreso de 20×20 mm y 75 mm de alto que sustituye al perfil 2020 (M5 desde abajo del piso 2, hoyo de 4.2 mm); luego se cambia a M4 con inserto de calor (`INSERTO_M4 = [5.6, 8]` mm, paso de 4.2 mm para el tornillo largo). Cabeza de cámara: ventana de 22 × 22 mm para el conector USB y el cable (antes 16), cuatro agujeros M2 en x, z = ±14 mm con tuerca atrás; después agujeros ovalados con 1 mm de juego y un marco que centra el PCB de la cámara (`CAM_PCB` + 1 mm, 0.5 mm por lado) con base 6 mm mayor. Se regeneran los STL de repisa, piso 2 y cabeza.
Problema.	No se tenía el perfil 2020 a mano (por el commit del pedestal, “impreso en lugar del perfil”). En la cámara, los agujeros exactos no coincidían con el PCB (por el “1 mm de juego”).
Solución.	Un pedestal impreso evita depender del perfil de aluminio. Se prefirió el inserto de calor al roscado directo en plástico. Los agujeros M2 se alargaron (hull de cilindros) para absorber el error de impresión; la ventana de 22 mm deja salir el conector USB y su cable.

Pruebas / evidencia.	Los cambios sucesivos de M5 (4.2 mm) a M4 con inserto (5.6 × 8 mm, con la nota “medir el suyo”) y de agujeros M2 exactos a 1 mm de juego son pruebas de montaje con las piezas reales. Según el equipo, el chasis y sus piezas pasaron por entre 20 y 50 iteraciones para verificar geometrías y tolerancias; no se guardan resultados de cada una.
Resultado.	Pedestal y cabeza imprimibles con tolerancias de montaje.
Commits.	5e485c1, 15d6883, 14a9821, d16b370, cc64a96

Etapa 19. Panel de cámara, cabeza en el CAD y motores invertidos (8 al 10-nov)

Qué cambió.

- El panel web (`/api/foto`) muestra la imagen de la cámara reducida a 160×120, calidad JPEG 40, con las regiones de pared dibujadas: rojo si ve pared, verde si está libre.
- Python separa `analiza(img)` de `leer_camara()` y protege la cámara con un `cam_lock`, porque ahora la usan a la vez el robot y la página.
- En el CAD, la placa de la cámara sube 12 mm (`CAM_DZ = 12`) para que su parte trasera quede libre. Se agregan costillas, un hueco para el conector USB arriba a la izquierda y un paso para los cables de alimentación abajo.
- Se actualizan manuales y PDF: panel de colores, vista de cámara, pedestal M4, cabeza nueva y hub con PD.
- En el sketch aparecen `INV_IZQ` e `INV_DER`, ambos `true`, para invertir cada motor por software.

Problema.	Sin una vista en vivo no había forma de ajustar a ojo las regiones de la cámara. La placa y el cableado de la cabeza no tenían espacio para el USB ni para la alimentación (deducido de los huecos que se agregan). Los motores giraban al revés; el comentario del código dice “en vez de voltear cables”.
Solución.	Vista de cámara con regiones, ajustes mecánicos de la cabeza e inversión de motores en el sketch.
Pruebas / evidencia.	Los commits no registran mediciones. La inversión de motores implica que se probó el robot y los motores giraban al revés.
Razonamiento.	Imagen de 160×120 con JPEG 40 para que el panel sea ligero y no compita con el control (deducido de la baja calidad elegida). Invertir por software se prefiere a voltear cables.
Resultado.	Las regiones se pueden afinar con la imagen del panel y los motores se invierten con una constante.
Commits.	3f72096, 1d5ae18, 3b2d8f5, d5c56ae

Etapa 20. La cámara como sensor de paredes con mapa 16×16 (10-nov)

Qué cambió.	`ROI` pasa de 3 regiones fijas a regiones con clave `"adelante,lado,pared"`: hasta 2 celdas adelante y por encima de la pared de enfrente (6 regiones, por ejemplo `"1,0,F": (0.35, 0.05, 0.65, 0.25)`). Esas paredes entran al mapa como "tentativas" para que el flood-fill descarte callejones antes de entrar. La cámara también pre-revisa el color del piso de la celda siguiente (`ROI_PISO = (0.30, 0.45, 0.70, 0.80)`, `HUE` por color, `FRAC_COLOR = 0.08`) y el TCS34725 confirma. Se rechaza un color del sensor si la cámara vio piso limpio en esa celda (falso positivo). Filtro de brillo para el piso negro: `BRILLO_MIN = 1.3`, es decir, una marca refleja al menos 1.3 veces más que el piso, tanto en Python como en el sketch. `META` se calcula según N: las 4 celdas centrales cuando N es par.
Problema.	Los ToF solo ven la celda actual, así que explorar es lento y se entra a callejones. La cámara, montada alta, puede ver por encima de las paredes. El sensor de color daba falsos positivos sobre el piso negro.
Solución.	Anticipar paredes y colores con la cámara y dejar que el sensor confirme.
Pruebas / evidencia.	El README dice que las regiones se ajustan viendo la imagen del panel (rojo/verde). No hay mediciones registradas.
Razonamiento.	Ya aquí hay una cautela: "Si la cámara se equivoca y bloquea todo, el robot ignora esas paredes tentativas" (README). Es la semilla de la etapa 21. La cámara solo adelanta; el sensor confirma.
Resultado.	El flood-fill usa información adelantada y hay un filtro de falsos positivos de color en dos capas (cámara y brillo).
Commits.	abb9ce3, 1b68e5b

Etapa 21. Laberinto 11×11, modo "tonto" y "los ToF mandan" (10-nov)

Qué cambió.	`N` pasa de 16 a 11 (celdas de 18 cm); con N impar la meta es una sola celda, la central. Aparece el modo "tonto" (pared derecha, sin depender del mapa) con selector en el panel; si da vueltas a una isla (misma posición y rumbo repetidos), cambia solo a modo mapa. El flood-fill pasa de BFS a Dijkstra (`heapq`) y una pared vista solo por la cámara ya no bloquea: cuesta `PESO_CAM = 3` celdas extra. Los VL53L0X se leen en cada celda, aunque ya se conozca, y antes de cada avance (`frente_libre()`); si ven pared al frente no se avanza. El mapa se corrige en los dos sentidos con `quita_pared`. Un mapa guardado de otro tamaño se ignora.
Problema.	El mensaje dice "Los VL53L0X mandan": la cámara bloqueaba rutas con paredes tentativas y podía equivocarse. Un mapa viejo o una pared falsa podían impedir llegar a la meta. No hay un dato numérico del fallo.

Solución. Jerarquía explícita: el sensor de distancia decide y la cámara solo encarece caminos. El modo "tonto" queda como respaldo simple.

Pruebas / evidencia. No hay mediciones. El cambio de N sugiere que se ajustó al laberinto real de 11×11 (deducido).

Razonamiento. Alternativas: bloquear (comportamiento anterior) o encarecer (nuevo). Se elige encarecer, porque una cámara equivocada solo alarga el camino y no lo cierra. Se acepta el costo de mantener Dijkstra en lugar de BFS para poder tener costos distintos. Se conserva el modo "tonto" como seguro contra fallos del mapa.

Resultado. El mapa ya no se queda bloqueado por la cámara y se autocorrige con los ToF.

Commits. 0aec4a2, c4e7291

Etapa 22. Odometría ligera y confianza en el ToF frontal (10-nov)

Qué cambió.

- dff8225, sketch: odometría ligera con medidas del CAD (`CELDA = 180`, `Y_LADO = 40`, `MEDIA_PARED = 6`). Cuando un ToF lateral pasa el borde de una pared (el poste entre celdas), corrige la posición y aprende la velocidad real (`escala`, acotada a 0.6 a 1.6). Con pared al frente a menos de `FRENTE_ODO = 400` mm corrige lo que falta con ganancia 0.2. Se ignoran correcciones de más de `MAX_CORR = 45` mm. Nuevo RPC `odo(i)` que lo expone al panel.
- dff8225, Python: la cámara mide la distancia a la base de la pared de enfrente con la geometría del CAD (`CAM_H = 162`, `CAM_Y = 32`, `CAM_INCL = 25`, `CAM_FOV_V = 75`) y ajusta cada avance hasta `ODO_MAX`.
- 70b44fa: `ODO_MAX` baja de 60 a 30 mm. Antes de avanzar se lee el frontal 3 veces y se toma la menor (`tof_frente()`). `celdas_libres()` limita cuántas celdas avanza y, si recorta la recta, marca la pared en el mapa. Si el frontal ve algo a menos de `TOF_RANGO = 1000` mm manda el frontal; la cámara corrige solo si el frontal no ve nada. `PESO_CAM` baja de 3 a 1. El panel agrega vista en vivo de los ToF.
- 76030d2: se deja de guardar `cal.jpg` y `mask.jpg` en `/app`.
- cc80dd9: se crea `PARED_F = 130` mm para el frontal: "de 140 para arriba no hay" pared.

Problema. La posición se llevaba solo por tiempo, y el tiempo deriva con la batería y el suelo. La cámara usada para corregir distancia resultaba menos fiable que el ToF (el commit 70b44fa le quita el mando, baja `ODO_MAX` a la mitad y baja `PESO_CAM`; el equipo confirma que la cámara metía mucho ruido). Guardar imágenes en `/app` hacía que App Lab fallara al arrancar (mensaje de 76030d2). El umbral lateral (120) no servía igual para el frontal: lecturas de 140 o más ya no eran pared.

Solución. Referencias físicas reales (bordes de pared y distancia frontal) para recorrer el reloj del avance, y el ToF frontal limita cuántas celdas se avanza.

Pruebas / evidencia. cc80dd9 trae una medición real: “a menos de 130 mm hay pared, de 140 para arriba no hay”. Los demás valores salen del CAD, no de pruebas de pista. `BORDE_AJ = 0` queda marcado “calibrar si corrige de más”.

Razonamiento. Se conserva la odometría por tiempo y se corrige “recorriendo el reloj”, en lugar de contar pulsos con encoders, que el robot no tiene (deducido). La cámara pierde autoridad commit tras commit: primero mide la pared, luego solo corrige “si el frontal no ve nada”. Se toma la menor de 3 lecturas: “una lectura mala no deja pasar”.

Resultado. Los avances se autocorrigen y nunca superan lo que el frontal ve libre.

Commits. dff8225, 70b44fa, 76030d2, cc80dd9

Etapa 23. Frenado, `D_FRENTE` y simulador “qué haría” (10-nov)

Qué cambió.

- 41063ab: `FRENA_K = 2` PWM por mm. Con pared al frente baja la velocidad al acercarse; si queda muy pegado (más de `D_CERCA = 15` mm por debajo de `D_FRENTE`) retrocede a `-VEL_MIN`, como máximo 0.8 s. El panel ToF “simula qué haría el robot” (`que_haria()`).
- e6e42fc: si hay pared enfrente y la distancia ya no baja durante 300 ms (con margen de 3 mm), se detiene (`dMin` más `tBaja`).
- 496e845: `D_FRENTE` pasa de 55 a 130 mm, `D_CERCA` de 15 a 10, y el frenado empieza a `D_FRENTE + 100` (230 mm). En el simulador, “Retrocede” se dispara por debajo de `D_FRENTE_TOF - 10`.
- 0f50bde: texto del simulador más claro (“gira 90 grados, no va de frente”).

Problema. Cambió con la prueba real: el comentario del código dice que el sensor lee de más y “nunca llega a 50”. Con `D_FRENTE = 55` el robot nunca cumplía la condición de parada y empujaba la pared; el mensaje de e6e42fc dice “no empuja la pared”.

Solución. Subir el umbral de parada a lo que el sensor realmente lee (130 mm), frenar en rampa antes de llegar y agregar un tope por tiempo: si ya no se acerca, se detiene.

Pruebas / evidencia. La lectura “nunca llega a 50 mm” con pared al frente es una medición real (comentario en el sketch). El simulador “qué haría” del panel sirvió para probar la lógica de decisión con lecturas ToF vivas sin mover el robot (“para pruebas, no mueve nada”).

Razonamiento. Se prefiere un número que el sensor sí alcanza (130) a uno geométrico (55) que no. El tope de 300 ms es una red de seguridad si la lectura se estanca. El simulador permite depurar la decisión sin riesgo.

Resultado. Para a 130 mm de la pared, retrocede si pasó y no empuja.

Commits. 41063ab, e6e42fc, 496e845, 0f50bde

Etapa 24. Solo seguir pared, prueba sin cámara y escritura atómica (10-nov)

Qué cambió.	285261b: se elimina el modo mapa de la estrategia. Queda solo el modo "tonto" con `seguir_pared(objetivos, mano)`: pared derecha y, si rodea una isla, prueba con la izquierda. Ya no guarda ni usa el mapa; sigue la pared con lo que leen los ToF en cada celda. 8833c86: `USAR_CAM = False` para una prueba sin cámara (solo ToF y sensor de color). c95da2f: los datos pasan a `/app/datos`; la escritura es atómica (archivo `.tmp` y `os.replace`) y la lectura es tolerante (`lee()` devuelve `None` si el JSON está roto).
Problema.	El mapa con flood-fill dependía de paredes que la cámara y los sensores veían mal. El mensaje dice "Solo modo tonto": se reduce el alcance; el motivo exacto no está escrito en el commit; coincide con las etapas 21 a 23, donde se descartó el mapa en favor de los ToF. Un JSON vacío tumbaba la app, y App Lab se confundía con archivos en la raíz de `/app`.
Solución.	Simplificar: seguir pared con ToF, aislar la cámara para probar y proteger los archivos de datos.
Pruebas / evidencia.	`USAR_CAM = False` sirve para aislar el comportamiento de los ToF y del sensor de color. La causa que nombra el comentario de la escritura atómica es el corte de energía: "si se apaga a la mitad no deja un archivo roto".
Razonamiento.	Una estrategia simple (pared) funciona sin mapa y por eso se probó primero. La cámara se apaga para saber qué parte de los fallos viene de ella.
Resultado.	Una app que arranca aunque haya un archivo roto y una estrategia mínima y comprobable.
Commits.	285261b, 8833c86, c95da2f

Etapa 25. Explorar hasta el amarillo; la cámara guía y cubre puntos ciegos (10-nov)

Qué cambió.

- a4f1221: nueva meta, encontrar el amarillo. `buscar_color()` explora sin fin y entre las salidas libres elige la celda menos visitada. Se detiene cuando el TCS34725 lee amarillo y, si hay cámara, ella también lo vio (si no coinciden, sigue buscando). Nuevo RPC `alto` (`pideAlto` en el sketch) para parar el avance en curso. D2 detiene la exploración. `USAR_CAM = True` otra vez.
- 7c1b4be: la cámara guía hacia el amarillo (`hacia_objetivo()` parte la mitad de abajo de la imagen en tercios y elige el que tiene más píxeles amarillos).
- 8c37fe8: se corrige el papel de la cámara: "Cámara solo desempata entre salidas igual de visitadas; los ToF y las visitas mandan".

- 38bd54c: la cámara cubre los puntos ciegos de los ToF. El frontal está en (0, 54) y los laterales en (±43, 40) con cono de unos 25 grados, así que las diagonales de enfrente no las ve nadie. `CIEGOS` define zonas (x ±25 a 60 mm, y 120 a 220 mm) y, si ve blanco más del 15% (`CIEGO_UMBRAL`), no avanza. Si el frontal pierde la lectura pero la cámara ve pared a menos de `CAM_CERCA = 230` mm, cuenta como pared. La foto se toma antes de avanzar.

Problema. La meta real es una marca amarilla, no una celda central. Los ToF no ven las diagonales de enfrente. Sin una regla de "guía", la cámara empujaba al robot a decisiones equivocadas: 7c1b4be pasó por encima de los ToF y 8c37fe8 lo corrigió un commit después.

Solución. Búsqueda abierta con la combinación sensor de color y cámara como doble confirmación. La cámara queda como referencia.

Pruebas / evidencia. Los puntos ciegos se calculan por geometría del CAD (posiciones y cono de 25 grados), no por mediciones. Que en 7c1b4be la cámara pasara a mandar y en 8c37fe8 se revirtiera muestra que se probó y no funcionó bien; el equipo confirma que con mucho peso a la cámara chocaba todo el tiempo.

Razonamiento. Doble confirmación del amarillo: evita parar por un falso positivo de un solo sensor. La jerarquía se decidió en dos pasos: primero la cámara manda, luego "los ToF y las visitas mandan"; se elige que la cámara solo desempate. Contar visitas en lugar de guardar paredes evita ciclos de islas sin depender de un mapa que pueda estar mal.

Resultado. El robot puede explorar sin conocer el laberinto y parar al encontrar el amarillo. Las diagonales quedan cubiertas por la cámara con geometría del CAD.

Commits. a4f1221, 7c1b4be, 8c37fe8, 38bd54c

Etapa 26. Robustez de la cámara y la web, colores de fábrica y manuales (11-nov, madrugada)

Qué cambió.

- cd5101b: los colores calibrados pasan a ser valores de fábrica. `REFS` y `refs[]` del sketch pasan de valores provisionales a PISO 0.282, 0.379, 0.333, 153; ROJO 0.602, 0.208, 0.188, 581; VERDE 0.155, 0.535, 0.309, 636; AZUL 0.16, 0.329, 0.508, 354; AMAR 0.409, 0.449, 0.142, 1836.
- 92cc120: la página reintenta abrir la cámara cada 5 s si se conectó tarde, y la reabre si se desconecta.
- 34f910f y 9e56d36: `bool` de Python en lugar de `numpy.bool` , y `limpio()` convierte tipos numpy antes de responder, porque FastAPI no los serializa.
- b5af071: busca la cámara hasta 30 s al arrancar. b881968: si no hay, sigue en segundo plano. bb1d40a: la búsqueda no bloquea el arranque de la app. 157c8d0: hilo `vigilar_cam` que revisa cada 2 s y reconecta la cámara si se desconecta; se quita el reintento de la página.

- 4ae53a0: manuales con búsqueda de cámara, alimentación del hub, colores de fábrica, web por WiFi y contraseña olvidada.

Problema. Al arrancar, a veces la cámara aún no estaba lista (el hub tarda). La lectura de colores con valores provisionales no servía. La API fallaba al serializar tipos numpy. La cámara fallaba por alimentación: el manual aclara que con la Q alimentada por VIN su USB-C no da 5 V al hub, así que el hub debe recibir energía por su puerto PD; alimentar solo la cámara con 5 V externos no basta. Según las pruebas del equipo, el UNO Q no siempre reconocía la cámara aunque tuviera 5 V y estuviera conectada; en la práctica se resolvió con un adaptador OTG "tonto" de USB-C a USB-A.

Solución. Un único hilo vigilante que busca y reconecta, en vez de reintentos repartidos en el arranque y en la página.

Pruebas / evidencia. Los colores de fábrica son lecturas reales del TCS34725 (el amarillo tiene brillo 1836 y el piso 153). El manual recoge una guía de diagnóstico: medir al menos 4.8 V en el hub, `adb shell lsusb`, `adb shell ls /dev/video*`. La secuencia de 4 commits (30 s bloqueante, segundo plano, sin bloquear, hilo vigilante) muestra la iteración sobre un fallo real. El test de alimentación sin PD de `docs/uno_q.md` (sección 12) figura como pendiente.

Razonamiento. Se descartó bloquear el arranque 30 s porque frena a la app entera; un hilo es mejor. Centralizar la reconexión evita dos lógicas compitiendo por `cam`.

Resultado. La app arranca siempre, la cámara se recupera sola y los colores quedan con valores calibrados por defecto.

Commits. cd5101b, 92cc120, 34f910f, 9e56d36, b5af071, b881968, 4ae53a0, bb1d40a, 157c8d0

Etapa 27. Alinear con giros de 5 grados y su retirada (11-nov)

Qué cambió.

- 8b497c4: `alinear()` barre de -10 a +10 grados en pasos de unos 5 (`MS_5 = 25` ms, RPC `ajustar`) y se queda donde la pared de enfrente se ve más cerca (perpendicular) o, sin ella, donde las dos laterales suman menos (paralelo al pasillo). Centrado con rumbo: `KD = 0.3` corrige según qué tan rápido cambia el error lateral (cada 100 ms). Mano derecha. El frente cuenta libre desde 220 mm (137 mm es pared en la celda propia: 0 celdas, no 1). La cámara se inclina de 25 a 40 grados y ve el piso desde unos 7 cm adelante del eje.
- 0c72165: se quita todo `alinear()` y `ajustar`. Se avanza derecho con `FRENTE_LIBRE = 150` mm. El centrado solo corrige si un lateral lee menos de `D_LADO_MIN = 90` mm; lejos de las dos paredes va derecho.

Problema. Los giros de 90 grados por tiempo (`MS_90 = 250`) dejan el rumbo chueco; el comentario lo dice: "el giro por tiempo deja el rumbo chueco". Alinear se descarta 15 minutos después. Según las pruebas del equipo, el robot intentaba

alinearse tomando muestras y girando poco (giros de 5 grados) y se volvió súper lento; por eso se quitó. Además el robot sí se desviaba en las rectas. El barrido son hasta 5 giros por celda, con un giro de 25 ms que podría no mover el robot (el README decía "si `MS_5` no alcanza a mover el robot, súbelo").

Solución. Primero alinear explícitamente; después, corregir solo cuando hay una pared cerca y dejar de corregir el rumbo a ciegas.

Pruebas / evidencia. El commit 0c72165 es una prueba negativa: lo que se probó en 8b497c4 no se mantuvo, porque dejaba al robot súper lento (pruebas del equipo). No hay lecturas registradas. La inclinación de 40 grados cambió una cifra del CAD (la cámara ve el piso desde 7 cm adelante del eje; `CIEGOS` pasó de "desde y = 116" a "desde y = 68").

Razonamiento. Alternativas: corregir el rumbo con barridos de ToF (descartado: lento y a lazo abierto) o no corregir salvo que haya riesgo de choque. Se elige la segunda por simplicidad. La banda muerta evita que el robot zigzaguee en un pasillo ancho. El problema de fondo, el rumbo, queda sin resolver hasta el MPU-6050 (etapa 31).

Resultado. Queda el centrado diferencial (`KC` y `KD`) con banda muerta de 90 mm. El frente libre pasa de 220 a 150 mm (luego vuelve a 220, etapa 30).

Commits. 8b497c4, 0c72165

Etapa 28. Reescritura: avance por distancia, cámara solo para amarillo y mano izquierda (11-nov)

Qué cambió.

- 9467a00: reescritura grande (146 líneas agregadas y 651 quitadas; `main.py` baja de unas 800 líneas). En el sketch, `mover(mm)` avanza por distancia con el frontal: al arrancar toma la lectura del frontal (`objetivo = dist[1] - total` , mínimo `D_FRENTE`) y para cuando bajó 180 mm (`CELDA`). Si el frontal no ve nada a menos de `RANGO = 1000` mm, avanza por tiempo (`MS_CELDA = 600`), y si aparece una pared a medio avance sigue por distancia. Se vuelve a parar a 130 mm y retrocede si queda más cerca. Se eliminan la odometría de bordes, `MS_RAPIDA` , `VEL_RAPIDA` , `escala` y `OFF_COLOR` . En Python se eliminan el mapa 11×11, el flood-fill, `ROI` , los puntos ciegos, la corrección de distancia con cámara, el pre-chequeo de piso y el modo "tonto". Python decide con 3 muestras de cada ToF; con 3 visitas a una celda (`ISLA`) toma la salida menos usada y la cámara desempata. La posición sale de los mm recorridos. La cámara solo busca amarillo (`HUE_AMAR = (20, 35)`) y el panel dibuja un recuadro donde lo ve.
- a6db9a1: regla de la mano izquierda (en `salidas()` el orden pasa de derecha, frente, izquierda a izquierda, frente, derecha). Se probó mano derecha y luego izquierda, y se priorizó ir de frente y después la mano

izquierda. Según las pruebas en pista, esa regla resultó peor: ver “Decisiones de ingeniería” (mano derecha pura).

Problema. Observado en pista: avanzando “1 celda por tiempo” (`MS_CELDA`) casi nunca llegaba a la posición de la celda, y el resultado cambiaba con el desgaste de la batería; los giros de 90 por tiempo (`MS_90`) casi siempre se quedaban cortos (el equipo estima que giraba unos 70 grados). Un `MS_90` proporcional sería de unos 320 ms (250 × 90/70), pero seguiría dependiendo de la batería, lo que justifica el MPU-6050. Los intentos de corregirlo (odometría de bordes con `escala`, corrección con cámara) añadían mucho código. Para quitarle las paredes a la cámara, según las pruebas del equipo: veía demasiadas paredes y el robot no sabía a cuál darle prioridad, a veces no avanzaba nada; como ve muy por arriba, veía más paredes de las relevantes, zapatos de la gente, etc., mucho ruido. Y cuando le dieron mucho peso a la cámara, chocaba todo el tiempo. La historia concuerda: cada commit desde la etapa 21 le quitó autoridad (bloqueaba, luego encarecía con `PESO_CAM` 3 y 1, luego desempataba, ahora solo busca amarillo), sus regiones `ROI` se ajustaban a mano y dependían de una inclinación que cambió de 25 a 40 grados, y los ToF ya decidían todo.

Solución. El sketch queda como ejecutor simple (mide, avanza, para) y Python como decisor de un paso a la vez. Con una pared al frente, el ToF mide la distancia real: bajar 180 mm es una celda. La posición en el mapa se calcula con los mm realmente recorridos (redondeo a celdas de 180 mm): un error menor de 90 mm no cambia de celda.

Pruebas / evidencia. El commit no cita mediciones. Los valores `FRENTE_LIBRE = 150` y `D_FRENTE = 130` vienen de lecturas anteriores (etapas 22 y 23). El problema de las celdas y los giros cortos es una observación del equipo en pista, sin mediciones por ciclo.

Razonamiento. Por distancia y no por tiempo, porque el tiempo depende de la batería y la fricción y el ToF frontal mide directamente lo que falta. El tiempo queda solo como plan B y como tope (`t > 3 * dur`). Se desecha la odometría de bordes porque la distancia al frente ya da la referencia. Compromiso: más simple y más robusto, a costa de renunciar a la anticipación por cámara; el mapa y la ruta rápida se pierden aquí y se recuperan, de forma más simple, en la etapa 29.

Resultado. Un diseño mucho más corto: unos 500 renglones menos en `main.py`.

Commits. 9467a00, a6db9a1

Etapa 29. Mapa de paredes y colores con ruta corta; cambios bruscos de ToF; cámara blanco/negro (11-nov)

Qué cambió.

- Mapa nuevo y ligero: `muros` (celda a [N, E, S, O]) y `colores`, más la `meta`, guardados en `/app/datos/mapa.json`. Los ToF anotan paredes; por donde pasó queda libre. Cada color que lee el TCS34725 (3 lecturas seguidas) se anota en la celda donde está el sensor (`Y_COLOR = 36` mm adelante del eje) y se cuenta una vez. La meta es la celda del amarillo.
- Si ya conoce la meta, la siguiente corrida va por el camino más corto conocido (BFS, `seguir_ruta`); si algo no cuadra, vuelve a explorar. Cada corrida empieza en la misma celda y mirando al mismo lado; si no, se usa el borrado largo del botón (2 s) que limpia mapa y conteo.
- Prioridad de salida: de frente, luego izquierda, luego derecha, luego media vuelta.
- Cámara blanco/negro: `ZONA_PASO = (0.30, 0.55, 0.70, 1.00)`, `BLANCO_MAX = 0.15`. Blanco es pared (el piso es negro y no lleva blanco). Si ve blanco en el camino, avanza lento (`VEL_LENTO = 100`) y el frontal hace el doble check: el sketch para a 130 mm si de verdad hay pared. La cámara nunca bloquea un camino, porque un reflejo o mala luz podría encerrar al robot.
- Cambio brusco de ToF (`SALTO = 120` mm, confirmado en dos lecturas seguidas): si se abre un lateral, para al centro de esa celda para escanearla. La idea es del equipo: si una lectura pasa de 50 a 200-300 mm (o de 100 a 300), algo cambió, por ejemplo se abrió un pasillo. En lugar de parar en seco a medio camino, avanza `TRAS_SALTO = 124` mm más (90 mm a centro de celda, más 40 mm que el sensor está adelante del eje, menos 6 mm de media pared; medidas del CAD), lo que además corrige la odometría. Si salta el frontal, vuelve a medir lo que falta (bug evitado: con la referencia de distancia vieja el robot seguía avanzando de más). Tras un salto lee 5 muestras en vez de 3 y va a la salida nueva que no esté en el mapa, la de lectura más lejana.

Problema. Sin mapa, cada corrida volvía a explorar desde cero. Con avance de celda completa se pasaban por alto aberturas laterales (puertas) a medio avance, porque el ToF lateral solo se leía al empezar; el cambio brusco las detecta en movimiento. La cámara de paredes ya no estaba (etapa 28); se le deja una función simple y robusta: ver si el camino inmediato es blanco o negro.

Solución. Un mapa pequeño y explícito, alimentado por sensores fiables (ToF y TCS34725), y la cámara reducida a una prueba binaria. BFS sobre ese mapa reemplaza al flood-fill con costos de la etapa 21.

Pruebas / evidencia. El README pide ajustar `ZONA_PASO` hasta que en un pasillo libre el recuadro salga verde. No hay mediciones registradas. `SALTO = 120` se justifica con los ejemplos del comentario, no con datos. Las pruebas automáticas de esta etapa se describen en la etapa 30.

Razonamiento. La cámara baja a “va lento” y el frontal decide (doble check): se evita que un falso blanco bloquee. Confirmar el cambio brusco en 2 lecturas evita que una lectura mala dispare una parada (“una lectura mala no”).

Resultado. La segunda corrida puede ir directo a la meta por el camino más corto conocido.

Commits. 1564598

Etapa 30. Máquina de estados, votos, frente libre a 220 mm, alto en amarillo desde el STM32 y web sin Bridge (11-nov)

Qué cambió.

- Máquina de estados en Python: `SENSA` a `DECIDE` a `GIRA` a `AVANZA` a `SENSA`, o `ESCANEA` en lugar de `SENSA` tras un salto. “Todo se decide con el robot quieto; el color se revisa todo el tiempo”. El STM32 conserva su máquina de tiempo real (NADA/AVANZA/GIRA). La fase sale en la OLED (`set_fase`) y en la web.
- Mapa con votos: cada lectura vota pared (+1) o libre (-1) con tope `VOTO_MAX = 5` ; por donde pasó queda `-VOTO_MAX` , libre seguro.
- `FRENTE_LIBRE` pasa de 150 a 220 mm.
- Alto en amarillo desde el STM32: el sketch define `AMAR = 4` y llama `termina()` en cuanto confirma ese color, sin esperar a Python.
- Durante una corrida la web no llama al Bridge: muestra la última lectura del robot (`vivo`) y la última foto (`ult_img`). La OLED ya no se refresca en movimiento (tarda unos 25 ms bloqueantes).

Problema.

- Una lectura errónea de ToF cambiaba el mapa de pared a libre o al revés (comentario: “una lectura mala no borra lo que vieron las demás”).
- 150 mm quedaba demasiado cerca del valor de pared en la celda propia. Con pared en su propia celda el frontal lee unos 130 mm; con la pared una celda más allá, unos 310 mm. Un umbral de 150 dejaba solo 20 mm de margen frente al ruido del sensor. Los laterales ya estaban bien: pared unos 50, libre más de 200, umbral de 120 a la mitad.
- Si la parada dependía del ciclo Python, el retraso hacía que el robot pasara del amarillo. Los motores no se afectan por la web (el control de 10 ms corre en el STM32), pero cada consulta web eran 5 a 9 llamadas al Bridge cada 150 a 300 ms compitiendo con el robot, y la foto de la web bloqueaba la cámara unos 130 ms. El riesgo real era que el “alto” en amarillo llegara tarde.

Solución. Estados explícitos, votos acotados, umbral al punto medio (220 mm), parada en el microcontrolador y una web que no compite con el robot.

Pruebas / evidencia. Las cifras de 130 y 310 mm son lecturas del frontal observadas antes (130 viene de la etapa 23; 310 y los laterales de las notas de la sesión). Pruebas automáticas en Python con datos simulados, sin robot: orden de salidas (frente,

	izquierda, derecha, media vuelta), umbral de 220 mm, ruta más corta BFS en un mapa de prueba, que una lectura mala no borre una pared vista 3 veces (votos), decisión tras un salto (salida nueva más lejana) y guardar/cargar el mapa. Todas pasaron. Se compiló el Python con `py_compile` en cada cambio. Esta versión todavía no se ha probado en la pista.
Razonamiento.	Votos con tope: se prefiere acumular evidencia a tomar la última lectura, y el tope evita que un mapa muy "seguro" no se pueda corregir. Parar en el STM32 elimina la latencia de Bridge y Python. Decidir quieto evita decidir con lecturas tomadas en movimiento. Umbral a mitad de camino: margen simétrico frente a lecturas bajas y altas. Orden de fuentes de error que se analizó en la sesión, de mayor a menor: 1) giros por tiempo (`MS_90`), que dependen de batería y piso (un giro de 80 grados tuerce todo lo que sigue), a resolver con el MPU-6050; 2) umbral de frente con poco margen, resuelto con 220 mm; 3) mapa que se sobrescribía, resuelto con votos; 4) odometría, porque la celda de 180 mm tolera hasta 90 mm de error y se re-ancla con la pared de enfrente y los saltos laterales.
Resultado.	Decisiones más estables, parada inmediata en el amarillo y un panel que ya no interfiere con la corrida.
Commits.	9635cf9

Etapa 31. MPU-6050 opcional: giros por ángulo y rumbo sostenido (11-nov)

Qué cambió.	Si un MPU-6050 (GY-521) responde en 0x68 al arrancar, el sketch gira hasta el ángulo, con giroscopio a ±500 grados/s (`LSB_DPS = 65.5`) y filtro de unos 44 Hz. Mide el sesgo (bias) con 200 muestras con el robot quieto al encender. El rumbo es absoluto: cada giro va al ángulo acumulado deseado, así que corrige lo que se pasó o faltó en el giro anterior. `ADELANTO = 6` grados corta el giro antes por la inercia. En recta sostiene el rumbo (`KG = 2.0` PWM por grado, tope ±20), además del centrado con paredes. El sentido del montaje (`signo`) se aprende en el primer giro, cuando el yaw supera 30 grados. Sin MPU el sketch sigue por tiempo (`MS_90`). `odo(8)` da el desvío en grados. La OLED muestra `G` si lo encontró y no se refresca en movimiento.
Problema.	Los giros por tiempo acumulan error de rumbo, el mismo problema de la etapa 27, que no se había resuelto. El UNO Q no trae IMU. Sin referencia angular, el robot tampoco corregía derivas en recta.
Solución.	Un giroscopio opcional y autodetectado: un solo sketch sirve con y sin él, y no hay que reprogramar al conectarlo.
Pruebas / evidencia.	Los commits no traen datos de pista. `ADELANTO` queda marcado "calibrar: si se pasa, sube; si le falta, baja". Es una constante pendiente de ajuste. El robot no tiene todavía el MPU-6050 conseguido (pendiente 8 de `docs/uno_q.md`).

Razonamiento.	Opcional, para no obligar a cablear el sensor y poder seguir probando sin él. Rumbo absoluto en lugar de giros relativos de 90 grados: no acumula error. Aprender el signo en el primer giro evita depender de cómo se monte la placa. Desactivar la OLED en movimiento prioriza el lazo de control sobre la pantalla.
Resultado.	Giros por ángulo y rumbo sostenido cuando el MPU está presente; sin él, el comportamiento anterior no cambia.
Commits.	6a6f10b

Etapa 32. Calibración de los sensores de distancia desde la página web (11-nov)

Qué cambió.	La página web suma una sección “Calibrar distancia”. Por sensor se guardan puntos (mm reales medidos con vernier contra la lectura cruda, promedio de 10) y se ajusta una recta `real = a × crudo + b` por mínimos cuadrados (con un solo punto, solo desfase). Se guarda en `/app/datos/tof.json` y se manda al STM32 (`set_tof`), que la aplica a cada lectura antes del control. `odo(9..11)` da la lectura sin calibrar para comparar.
Problema.	Los VL53L0X leen con un desfase propio por sensor (“nunca llega a 50” con pared al frente) y varían ±15 mm según las pruebas del equipo; los umbrales se habían ajustado sobre lecturas crudas.
Solución.	Calibración de 3 puntos (50, 100 y 150 mm con vernier), más 250 a 300 mm para el frontal con flexómetro, porque sus umbrales viven entre 220 y 310 mm y extrapolar la recta más allá de lo medido es donde más se equivoca.
Pruebas / evidencia.	Prueba automática del ajuste con datos simulados (sin puntos, 1 punto y 3 puntos). Pendiente: calibrar en el robot y volver a medir los umbrales con lecturas calibradas.
Razonamiento.	Hacerla desde la web siguió el patrón que mejor funcionó en el proyecto: la calibración de colores desde el celular fue, según el equipo, de lo más útil. Calibrar sin reprogramar permite repetirla en la pista en minutos.
Resultado.	Cada sensor reporta milímetros reales con el mismo procedimiento que los colores.
Commits:	b416c9f.

Etapa 33. Dos versiones del sketch: con y sin MPU-6050 (11-nov)

Qué cambió.	Se conserva `uno_q/sketch/` (con MPU-6050 autodetectado) y se agrega `uno_q/sketch_sin_mpu/` , la misma lógica sin el código del giroscopio (giros y rectas solo por tiempo).

Razonamiento. La versión con MPU ya funciona sin él, pero tener la versión sin MPU por separado da un respaldo simple y fácil de leer mientras se consigue y calibra el giroscopio, y permite comparar en pista los dos métodos de giro.

Pendiente. Compilar ambas en App Lab y confirmar en pista `MS_90` (~320 ms, porque con 250 ms giraba ~70°) y el cambio a mano derecha pura.

Commits: 5906d64.

Etapa 34. Cámara monocromática OV9281: solo blanco y negro (11-nov)

Problema. La cámara real del robot es una innomaker U20CAM-9281M: sensor OV9281 de 1 MP, **monocromático**, con obturador global y hasta 120 fps. Sin filtro de color, la búsqueda de amarillo por tono (HSV, `HUE_AMAR`) nunca podía activarse: la saturación de una imagen en gris es cero.

Razonamiento. El amarillo por cámara solo desempataba islas; la meta siempre la confirmó el TCS34725, el componente más confiable según las pruebas en pista. Mantener código que nunca se activa confunde a quien lo lee. Lo que sí se usa, distinguir blanco (pared) de negro (piso), depende de la luminancia, y un sensor monocromático la mide mejor: cada píxel recibe toda la luz en lugar de un tercio, con menos ruido. El obturador global además evita que la imagen se deforme con el robot en movimiento.

Cambio. Se quitan `hacia_objetivo`, `HUE_AMAR`, `SAT_MIN`, `VAL_MIN` y `FRAC_COLOR`, el desempate por cámara en islas y el recuadro amarillo de la web. El blanco pasa de un rango HSV a un umbral de gris (`BLANCO_MIN` = 170) que acepta la imagen en 1 o 3 canales. Se actualizan BOM, CAD (medida de la placa por confirmar) y manuales.

Pruebas. Pruebas automáticas del Python (decisiones, votos, BFS, guardar/cargar) y de `frac_blanco` con imágenes sintéticas: piso negro da 0, una franja blanca en la zona de paso da 0.37. Falta probarlo con la cámara real.

Etapa 35. Exprimir la cámara monocromática: rumbo, umbral automático y 2 técnicas experimentales (11-nov)

Problema. La cámara solo daba un dato (hay blanco o no) y con un umbral fijo que dependía de la luz. Mientras, dos problemas de pista seguían abiertos: el robot se desviaba en recta y no veía las esquinas en diagonal (por eso se habían pensado ultrasónicos a 45°). Además, la inclinación de la cámara nunca se corrigió: quedó la del primer soporte (la última versión del CAD no se imprimió).

Razonamiento. Una escena de piso negro y paredes blancas tiene un histograma bimodal (Otsu lo separa solo) y bordes muy marcados. Las bases de las 2 paredes de un pasillo son rectas paralelas en el piso; en la imagen se cortan en el punto de

fuga, cuya columna depende solo del ángulo del robot respecto al pasillo, no de su posición lateral ni de la inclinación de la cámara. Eso da una medida **absoluta** de rumbo, que el giroscopio no tiene (integra y deriva). Para no depender de la inclinación sin corregir, todo se calibra con el robot real desde la web, y la distancia por cámara usa el ToF frontal como regla.

Cambio.

- Python estable (`python/`): exposición fija desde la web, umbral de Otsu acotado a [100, 220] con filtro de mediana, y rumbo por punto de fuga con rectas ajustadas por mínimos cuadrados con rechazo de atípicos. Una foto por avance.
- Sketches: nuevo RPC `ajustar(g)`. Con MPU y sentido aprendido no gira: mueve el rumbo deseado y el control de recta (`KG`) lo corrige al avanzar (la cámara corrige la deriva del giroscopio). Sin MPU gira en su lugar por tiempo. `GIRA` pasa a trabajar en grados (`girar(k)` manda $k \times 90$), y el corte por inercia se escala para giros chicos.
- Python experimental (`python_experimental/`, mismo código con `EXPERIMENTAL = True`): distancia a la pared en 5 columnas (calibrada fila → mm con el frontal) y aberturas laterales en la celda siguiente (avance lento y voto débil de libre).
- Web: "Calibrar cámara" (exposición, "Está derecho", "Fijar escala", y puntos de distancia en la experimental); la imagen dibuja bases de paredes, punto de fuga y, en la experimental, la base por columna.
- Quedan 4 combinaciones: sketch con o sin MPU × Python estable o experimental.

Pruebas.

Con imágenes sintéticas de un pasillo generadas por proyección de una cámara a 150 mm de altura y 35° hacia abajo, con ruido: el rumbo medido coincidió con el real dentro de ±0.1° de −12° a 12°, también descentrado 25 mm y con una pared abierta; la distancia central dio 209/306/400/517 mm con paredes a 200/300/400/500 mm; las aberturas izquierda, derecha y con el robot girado 6° se detectaron sin falsos positivos. Se revisó la web con datos simulados (Playwright). Siguen pasando las pruebas de decisiones, votos, BFS y guardar/cargar. Los sketches no se han compilado (pendiente en App Lab) y nada de esto se ha probado con la cámara real.

Etapa 36. Umbrales del frontal en milímetros reales (11-nov)

Problema.

Al calibrar en pista con una pared a 100 mm, el frontal leyó 187 mm (antes 192): mide unos 87 mm de más. Los laterales salieron casi exactos (−5 y −0.2 mm). Los umbrales del frente (`D_FRENTE` y `PARED_F` en 130, `FRENTE_LIBRE` en 220) se habían ajustado con lecturas sin calibrar; con la corrección aplicada, el robot habría frenado una celda antes y confundido la pared de la celda siguiente con un frente libre.

Razonamiento. El desfase explica una observación vieja: "el sensor lee de más, nunca llega a 50". Con el robot centrado, el frontal queda a 90 − 6 − 40 = 44 mm reales de la pared de su celda, y 130 sin calibrar ≈ 43 reales. Pasar los umbrales a milímetros reales deja el código igual para cualquier sensor calibrado, en vez de depender del error de un sensor en particular.

Cambio. Sketches: `D_FRENTE` y `PARED_F` 130 → 45. Python: `PARED_F` y `D_FRENTE_TOF` 130 → 45, `FRENTE_LIBRE` 220 → 135 (a la mitad entre ~45 y ~225). Los laterales no cambian. **Requiere el frontal calibrado:** si se borra su calibración, el robot se pegaría a la pared.

Etapa 37. Mano izquierda pura, compensación de motores y alto a 10 mm (11-nov)

Observación en pista. Los colores se detectan muy bien. El robot se va un poco a la izquierda: la llanta izquierda avanza menos.

Cambio.

- **Regla de la mano izquierda pura:** izquierda, frente, derecha, media vuelta. Corrige lo visto antes: con "frente primero" el robot se atoraba en una zona, porque un seguidor de pared solo es completo si siempre prefiere el lado de su mano.
- `AJUSTE_DER` **= −10:** en recta le da más PWM al motor izquierdo. Es un valor de partida; si sigue yéndose a la izquierda se baja (−15, −20), y si se va a la derecha se sube.
- `D_FRENTE` **= 10 mm reales** (antes 45), a pedido del equipo, con el frontal ya calibrado. `D_CERCA` = 5 mm. `FRENTE_LIBRE` = 110 mm y `D_FRENTE_TOF` = 10 mm en Python.

Riesgo anotado. Con el robot centrado, el frontal queda a unos 44 mm de la pared de su celda (90 − 6 − 40). Frenar a 10 mm deja el eje unos 34 mm después del centro de la celda. Y por debajo de unos 30 mm el VL53L0X pierde exactitud. Hay que verificar en pista que no toque la pared y que la siguiente decisión siga saliendo bien.

Etapa 38. Exploración completa: la meta al centro (11-nov)

Problema en pista. El robot no encontraba la meta. La meta está al centro del laberinto y solo tiene 2 entradas.

Diagnóstico. Las paredes que rodean el centro no tocan el muro exterior: forman una isla. Un seguidor de pared solo recorre el muro conexo al que está pegado, así que con mano izquierda (o derecha) rodea la isla sin entrar nunca. La regla anterior de "3

	visitas → salida menos usada" no basta: el robot casi nunca repite la celda exacta.
Cambio.	Exploración tipo Trémaux / búsqueda en profundidad: prefiere salidas hacia celdas sin ver, en orden de mano izquierda; si no hay, BFS por el mapa conocido a la celda más cercana con lados sin explorar. Se quitan la regla de islas y la de "salida más lejana tras un salto" (ahora también va a la izquierda si es nueva, como pidió el equipo). Las celdas vistas se guardan con el mapa.
Pruebas.	Simulación con 30 laberintos de 9 × 9 con la meta en una isla central de 3 × 3 con 2 entradas: mano izquierda pura 0/30, exploración completa 30/30. Pruebas unitarias de `decidir()` y `frontera()`.
Otros cambios de la misma etapa.	`D_FRENTE` = 35 mm (pedido: 30 a 40), `PARED_F` = 55, `FRENTE_LIBRE` = 130. Cámara: estiramiento de contraste antes de Otsu y sensibilidad ajustable desde la web.

Etapa 39. Primera corrida larga con MPU: 3 fallas del log (11-nov)

Del log de pista (157 s, sin cámara conectada):

1. **"Avanzo 0 mm" después de girar.** El robot giraba y mandaba avanzar en seguida. El sketch arrancaba el avance con la lectura del frontal de **antes** del giro (la pared que tenía enfrente), creía que ya estaba a 35 mm y no se movía. Python volvía a leer, ya mirando al nuevo lado, y decidía otro giro sobre el anterior. **Cambio:** 0.1 s de espera después de cada giro para que los ToF den una lectura nueva.
2. **Ida y vuelta entre 2 celdas** (4 medias vueltas seguidas entre (−6,0) y (−5,0)). Al regresar descentrado, el lateral leía 106 mm en el paso por donde había llegado (umbral 120): todo parecía cerrado y daba media vuelta. **Cambio:** un lado por donde el robot ya pasó (voto ≤ −3) cuenta como libre aunque un ToF diga pared. Si de verdad es pared, 2 lecturas más lo corrigen.
3. **Los 2 laterales en 9999** (sin lectura) varias celdas seguidas: lo más probable es que haya salido del laberinto por la entrada. **Cambio:** si los 2 laterales no ven nada a 2 m (`FUERA`), marca esos lados como pared y regresa.

Además, la cámara ya no se busca en `/dev/video0` y `video1` (son internos) y OpenCV queda en nivel ERROR: se acaban los WARN cada 2 s.

Pruebas. Simulación de 30 laberintos con la meta en una isla central, con 15 % de lecturas libres leídas como pared: todos llegan a la meta, y en la mayoría con menos pasos que antes.

Etapa 40. Cronómetro, registro de colores y alineación con MPU (11-nov)

Pedido del equipo.

Ver en cuánto tiempo encuentra el amarillo, qué colores fue viendo, y aprovechar el MPU para enderezarse mejor.

Cambio.

- **Cronómetro:** la web muestra el tiempo de la corrida en vivo, a qué segundo encontró el amarillo y, al terminar, la duración total. También queda en el log de Python.
- **Registro de colores:** cada color confirmado por el TCS34725 se anota con su segundo y su celda; la web muestra los últimos 15.
- **Alineación con MPU (`alinear()`):** antes de cada avance, si el rumbo se desvió 3° o más del deseado (hasta 45°; más se toma como lectura mala), gira en su lugar hasta corregirlo. Así no empieza el avance chueco, que es cuando las esquinas tocan la pared. En el sketch sin MPU la función existe y no hace nada, para que el Python sea el mismo.

Etapa 41. El MPU completo: choques, atorones, giros verificados y robot levantado (11-nov)

Pedido del equipo.

Usar todo lo que da el MPU-6050, no solo el giroscopio para girar.

Cambio (sketch con MPU; el sin MPU responde 0 a todo esto).

1. **Choques con el acelerómetro.** Cada 10 ms lee la aceleración horizontal y le resta un promedio lento, que quita la inclinación del montaje. Un golpe de más de 0.6 g en un avance, después de los primeros 150 ms (el arranque), es un choque: retrocede 250 ms y termina el avance. Python lo registra con la fuerza del golpe, vuelve a leer los ToF y se endereza antes de seguir. El golpe máximo de cada avance queda en `odo(15)` para calibrar `CHOQUE_G`.
2. **Giro atorado.** Si girando pasan 400 ms sin que el giroscopio vea 2° de giro, una esquina está trabada contra la pared: retrocede 200 ms y sigue girando (hasta 2 veces).
3. **Giros verificados.** Después de cada giro, Python lee el desvío respecto al rumbo deseado. Si pasa de 10°, lo termina girando en su lugar (`alinear(180)`, 2 intentos). Si aun así quedó 45° o más fuera, acepta el rumbo real: corrige el rumbo del mapa en cuartos de vuelta y mueve la referencia del sketch, para que el resto del mapa no se tuerza.

4. **Robot levantado.** Si la aceleración vertical baja de 0.8 g durante 150 ms (inclinado o en el aire), detiene los motores y Python termina la corrida, porque la posición ya no vale.

No se hizo: giros en curva. Hoy el robot decide quieto en el centro de cada celda; una curva termina fuera del centro y rompe el mapa por celdas. Queda para cuando lo demás esté estable.

Pruebas. Revisión de sintaxis de los 2 sketches con g++ y cabeceras de Arduino simuladas. Pruebas de Python: verificación de giro (desvío de 95°, 92° y 88° → acepta un cuarto de vuelta y mueve la referencia 2°), control web, decisiones y mapa. Los umbrales (0.6 g, 0.8 g, 400 ms) son de partida: hay que calibrarlos en pista con el golpe que muestra el log.

Etapa 42. Un solo sketch, con MPU-6050 (11-nov)

Decisión del equipo. El robot ya lleva el MPU-6050 montado y funcionando (la OLED muestra `G`). Tener 2 sketches (`sketch/` y `sketch_sin_mpu/`) confundía: había que mantener los dos iguales y decir cuál cargar. Se retira `sketch_sin_mpu/` del repositorio; la última versión queda en el commit `2e01996` y se describe en la sección “Control de versiones” de los manuales. `sketch/` conserva el respaldo: si el MPU no responde al arrancar, gira y avanza por tiempo.

Documentación. Manual técnico con la sección 6.1 sobre el MPU-6050 (configuración de registros, integración del rumbo, alineación, verificación de giros, choques, atorones, robot levantado, fusión con la cámara y calibración); manual de uso con la tabla de ajustes del MPU y fallas; diagrama de conexiones y BOM con el MPU-6050 (AD0 a GND); sección “Control de versiones” en los 2 manuales.

Etapa 43. Un solo Python (11-nov)

`python_experimental/main.py` era el mismo archivo que `python/main.py` con `EXPERIMENTAL = True`. Tener 2 copias obligaba a mantenerlas iguales y confundía al ver el repositorio. Se retira la carpeta (queda en el commit `2e01996`); la versión experimental se activa cambiando esa línea.

Etapa 44. Centrado en el carril con la geometría del chasis; choques (11-nov)

Del log de pista. La exploración ya encontró el amarillo (113 s en una corrida). Pero en otra el robot se quedó decenas de veces en la misma celda: "choque a los 0 mm" con el frontal libre a 300-450 mm y un lateral a 40-80 mm. Estaba trabado con una esquina contra una pared lateral: retrocedía 250 ms a PWM 80 (muy poco), se enderezaba al mismo rumbo y volvía a chocar, unos 15 s por punto. Además, un avance llegó a 895 mm porque varios saltos laterales seguidos lo alargaron.

Diagnóstico.

1. **Centrado.** Solo corregía con un lateral bajo 90 mm (`D_LADO_MIN`): con una sola pared el robot quedaba a 90 mm de ella, unos 49 mm fuera del centro, y en el siguiente pasillo arrancaba pegado a una pared.
2. **Rumbo del MPU contra las paredes.** Si el giroscopio derivó, el control de recta (`KG`) sostiene un rumbo que no es el del pasillo, y el centrado lo pelea: el robot avanza en diagonal hacia una pared.
3. **Distancias "a ojo".** `D_LADO` = 50 y `D_FRENTE` = 35 no venían de la geometría.

Cambio.

- **Geometría del CAD** (`MEDIA_CELDA` = 84 mm del centro de la celda a la cara de la pared; frontal 54 mm adelante del eje; laterales a 43 mm del centro y 40 adelante): `D_FRENTE` = 84 − 54 = **30 mm**, `D_LADO` = 84 − 43 = **41 mm**, `TRAS_SALTO` = 90 + 40 − 6 = 124 mm, `Y_COLOR` = 34 mm (`COLOR_Y`), `FRENTE_LIBRE` = 120 mm (a la mitad entre ~30 y ~210).
- **Centrado en el carril:** con pared a un lado (< 120 mm) se centra contra 41 mm; con las 2, que lean igual.
- **Deriva del MPU con las paredes:** con las 2 paredes, centrado y paralelo durante 300 ms, la mitad de la diferencia entre el rumbo real y el deseado se corrige como deriva.
- **Choque:** retrocede 450 ms a PWM 110. Python se aleja de la pared más cercana (gira 20°, avanza 50 mm, unos 17 mm de lado, y regresa al rumbo).
- **Avance máximo** de 304 mm (una celda + `TRAS_SALTO`).
- El log ya no muestra "ERROR al arrancar" cuando App Lab detiene la app (código 143).

Pruebas. Sintaxis del sketch con g++ y cabeceras simuladas; pruebas de Python (maniobra de carril, decisiones, mapa, control web) y simulación de la meta al centro (30/30). Falta probar en pista.

Etapa 45. Choques confirmados (11-nov)

Del log de pista. Con el umbral de 0.6 g el robot "chocaba" sin chocar: golpes de 0.75 a 1.5 g en avances normales (vibración de los motores y del arranque), y cada falso choque lo hacía retroceder y hacer la maniobra de carril aunque estuviera a 60-87 mm de la pared.

Razonamiento. Un golpe por sí solo no distingue un choque de una vibración. Lo que sí lo distingue es la consecuencia: si chocó, deja de avanzar.

Cambio. El golpe (> 0.8 g) solo abre una ventana de 250 ms; es choque si en ella el frontal no bajó 10 mm. Sin pared enfrente para medirlo, solo cuenta un golpe de más de 2.5 g. La maniobra de carril solo se hace si un lateral quedó a menos de 35 mm (centrado lee ~41).

Etapa 46. Pasillos de un tirón; golpe a 1.5 g (11-nov)

Del equipo. El robot se veía tieso: paraba, leía, decidía y arrancaba en cada celda. Además, el equipo pidió subir el umbral de golpe a 1.5 g.

Razonamiento. El sketch ya vigila los ToF durante el avance y para en cuanto se abre un lateral. En un pasillo (pared a ambos lados) no hay nada que decidir, así que parar en cada celda no aporta información.

Cambio. Con pared a ambos lados y yendo de frente, Python pide hasta `PASILLO` = 3 celdas de un tirón. Si se abre un lateral, para en el centro de esa celda como antes; las celdas intermedias se anotan con pared a los 2 lados. El tope de saltos del sketch ahora es lo pedido + `TRAS_SALTO` (antes, una celda + `TRAS_SALTO`). La simulación sigue en 30/30 con ~9 % menos paradas. `CHOQUE_G` pasa de 0.8 a 1.5 g: en el último log los falsos choques fueron de 0.75 a 1.52 g.

Etapa 47. Exploración hacia el centro, sin cámara (12-nov)

Del equipo. Fotos del laberinto desde los 4 lados: 11 × 11 celdas de 18 cm, la meta amarilla al centro, la salida en una esquina (al azar). Las cintas de colores solo hay que leerlas. Por ahora sin cámara.

Razonamiento. No se precarga un mapa de las fotos: con la perspectiva no se ubica cada pared con precisión de celda, una pared mal puesta hace chocar al robot y la pista puede cambiar. Lo que sí sirve: la meta está a 5 celdas de cualquier esquina en cada eje. Las fotos confirman que el cuarto central es una isla, así que un seguidor de pared no entraría.

Cambio. `USAR_CAM = False`. Entre las salidas a celdas nuevas, la que más acerca al centro (empate: izquierda, frente, derecha); `frontera()` escoge menos pasos + distancia al centro. El lado del centro se deduce en la esquina de salida (sus

	lados libres apuntan hacia adentro) o con el primer movimiento en cada eje (`ubica()`). Simulación, 40 laberintos de 11 × 11 con isla central y esquina al azar: 40/40, 1173 pasos en total contra 5479 antes (peor caso 93 contra 163).
Pendiente.	El robot avanza 15-16 cm por celda en vez de 18: falta saber si pasa con pared lejos (más de 1 m: avanza por tiempo, subir `MS_CELDA`) o cerca (lectura del frontal).

Etapa 48. Simulación 2D: reglas de ToF y MPU (12-nov)

Del equipo.	Simular 200 laberintos con la forma de la pista e iterar qué reglas de los ToF y del MPU sirven.
Cómo.	Simulador 2D (`uno_q/sim/`): robot de 120 mm con motores, ToF con ruido, fallas y cono, giroscopio con deriva, golpes de vibración como los de los logs y choques por contacto. Usa las funciones reales de `main.py`. Calibrado con la pista: 600 ms = 154 mm (lo que reportó el equipo) y 250 ms de giro ≈ 70°. 200 laberintos de 11 × 11 con isla central, salida en una esquina al azar.
Hallazgos.	El código anterior llega en 26 %: el robot pierde la celda. Bug real del sketch: al pedir un avance con la pared más cerca que lo pedido (o pegado a ella), paraba pero reportaba los 180 mm completos y Python se adelantaba una celda. Los umbrales de los ToF (`PARED`, `FRENTE_LIBRE`, `SALTO`, muestras, votos) no cambian nada dentro del ruido. Lo que más pesa: el bug (sin corregirlo 67 %), `centrar` (sin él 58 %), `KD` = 0.3 torcía el rumbo (59 %) y el MPU (sin él, giros por tiempo, 52 %). El detector de choques confirmado no cambia la tasa con vibración normal; con vibración alta, contar solo el golpe baja a 63 % con casi 10 falsos choques por corrida.
Cambio.	Sketch: cuenta solo lo que cabe hasta la pared; nuevo `centrar(lim)` (tras cada avance, con pared enfrente a menos de 600 mm, queda a 30 mm + k celdas de ella); `KD` 0.3 → 0; `TRAS_SALTO` 124 → 110; `RANGO` 1000 → 600; `MS_CELDA` 600 → 700. Python: pide `centrar` tras cada avance (`CENTRA_F` = 600) y, tras un choque sin pared cerca (un poste), esquiva 15° a un lado. Resultado: 90 % (85 % con vibración alta). Detalle en el manual técnico, 9.4.
Pendiente.	Probar en pista; si `centrar` va y viene frente a la pared, bajar `CENTRA_F` o subir la tolerancia (4 mm).

Pruebas realizadas

EN CORTO

Lo medido o probado, separado de lo deducido. El equipo probó en pista todo el desarrollo; la última versión (distancia, saltos, votos, máquina de estados, MPU) todavía no.

La tabla distingue lo medido o probado de lo que solo se deduce de la secuencia de commits (marcado **deducido**). Cuando el repositorio guarda la sucesión de valores pero no el resultado de cada intento, se dice. Las pruebas en pista del equipo están en su propia subsección; la última versión (distancia, saltos, votos, máquina de estados, MPU) aún no se ha probado en pista.

Pruebas en pista

Datos de la pista: laberinto de prueba de 2 m x 2 m en total, con paredes de madera. La batería dura unas 3 horas con carga al 100 % haciendo pruebas constantes. Ninguna pieza se rompió. Fuente: respuestas del equipo del 11-nov.

Prueba	Qué observamos	Qué cambiamos por ello
1. Avance por tiempo (`MS_CELDA`)	Casi nunca llegaba a la posición de la celda; cambiaba con el desgaste de la batería	Avance por distancia con el ToF frontal; el tiempo queda de plan B
2. Giros de 90 por tiempo (`MS_90`)	Casi siempre faltaba: el equipo estima que giraba unos 70 grados	Giros por ángulo con el MPU-6050 (un `MS_90` proporcional sería de unos 320 ms, pero seguiría dependiendo de la batería)
3. Rectas	Sí se desviaba. Intentaron alinear con muestras y giros de 5 grados, pero el robot se volvió súper lento	Se quitó `alinear()` ; centrado solo con laterales y rumbo sostenido con el MPU
4. Choques	Cuando se dio mucho peso a la cámara, chocaba todo el tiempo	Los ToF mandan; la cámara no bloquea ni decide paredes
5. ToF	Los 3 varían poco, unos 15 mm de ruido a cada lado; nunca leyeron 0 ni lecturas raras	Umbrales con margen (frontal 220 mm, lateral 120 mm); posible offset por sensor
6. Sensor de color TCS34725	Es el mejor componente: ya calibrado es muy bueno	Se confía en él para amarillo y marcas; el STM32 para al confirmar amarillo
7. Cámara para paredes	Veía demasiadas paredes y no sabía a cuál darle prioridad, a veces no avanzaba; ve muy por arriba y captaba paredes lejanas, zapatos de la gente y mucho ruido. También costó configurarla para verla en la página web	Se quitó para paredes; queda para amarillo y blanco/negro. Idea futura: calibrar exposición y parámetros

Prueba	Qué observamos	Qué cambiamos por ello
8. Alimentación de la cámara	El UNO Q no siempre reconocía la cámara aunque tuviera 5 V y estuviera conectada	Adaptador OTG “tonto” de USB-C a USB-A
9. Laberinto de prueba	2 m x 2 m en total, paredes de madera	Se prueba en esa pista; verificar que las paredes sean blancas para la cámara
10. Meta amarilla	La encontraba y se estacionaba bien; entre más grande la zona de color, mejor, porque el sensor RGB podía pasar por un lado y saltarse una línea amarilla delgada	Zonas de color más grandes
11. Mano derecha vs izquierda	Con la mano derecha llegó más lejos y se movía más; con la izquierda se quedaba atorado en la misma zona	Volver a la mano derecha pura (pendiente en el código)
12. Batería	Unas 3 horas con carga al 100 % en pruebas constantes	Ninguno; autonomía suficiente
13. Piezas	Ninguna se rompió	Arduino vertical con cinchos para conservar los 12 cm y que el OTG no salga del contorno
14. Chasis	Entre 20 y 50 iteraciones del chasis y sus piezas para verificar geometrías y tolerancias	Geometrías y tolerancias finales (hueco del motor 11.9 mm, soporte ToF, etc.)

Mediciones e iteraciones físicas

Prueba	Cómo	Resultado	Qué se cambió por ella
Hueco de la cuna del motor	Medición del motor (12 × 11 mm) y cinco impresiones sucesivas (deducido; el repositorio no guarda el resultado de cada una)	Hueco de 11.94 + 0.1 por lado, 12, 12.05, 11.85 y finalmente 11.9 mm; ajuste a presión	`MOT_A = 12`, `MOT_H = 11`, hueco fijo de 11.9 mm en lugar de holgura calculada
Ranura del soporte del ToF	Impresión real del sensor suelto; el comentario del CAD dice que con 2.0 mm “bailaba” (inferido del comentario)	Ranura de 2.0 mm floja	`RAN_TOF = 1.8` mm y postes que abrazan 4 mm de cada oreja
Tornillos del soporte del ToF	Verificar el agujero real de la oreja del sensor y el acceso a la cara trasera en los laterales	La oreja pasa M3, no M2; la columna del piso 2 tapa la cara trasera y no se puede roscar	M2×8 a M3×8, luego bloque detrás del módulo con 2 insertos y M3×10 por el frente
Medidas del módulo VL53L0X	Medición directa y plano de UNIT	Alto 11 mm (no 10.7); agujero a 5.5 mm del borde de los pines; el agujero se subió 1.5 mm después	`TOF_AG = [20, TOF[1] - 5.5]`; bloque con pared de 2.2 mm (de +6 mm de ancho a +3.4 mm); muesca de 9 a 10 mm
Medidas de rueda, motor y pila	Medición física	Rueda D 45 × 17, motor 26.11 × 11.94 x 11.94, pila 70 mm	Pozo de rueda, postes delanteros a (±34, 55), `TOF_LADO` [43, 40]
Piso 2 ya impreso	Comparar el modelo con la pieza impresa	Ranuras en Y = 34.5 y postes de 19 mm	Se adaptó el modelo a la pieza; solo cambian 2 agujeros delanteros
Pieza de prueba de tolerancias	`PIEZA="prueba"`: holgura 0.2 mm por lado, M3 pasante 3.4, M3 a rosca 2.8, M2 a rosca 1.8	No hay evidencia de que se imprimiera en esa etapa	Valores de partida de la tolerancia FDM
Montaje del pedestal y la cabeza de cámara	Cambios sucesivos de M5 (4.2 mm) a M4 con inserto 5.6 × 8 mm, y de agujeros M2 exactos a ovalados con 1 mm de juego; según el equipo, entre 20 y 50 iteraciones del chasis y sus piezas	No se guardan resultados de cada una	Pedestal impreso con inserto M4; marco que centra el PCB con 0.5 mm por lado
Rutas de cables y choques de conectores	Verificación geométrica por intersección en OpenSCAD (37 cables)	Sin choques tras los ajustes	Batería sobre el eje, pasillos de 27 mm, topes de batería fuera de los pasillos
Memoria del ATmega	Compilación y reporte de uso	Arduino: 18.9 KB flash (58%) y 1.47 KB RAM (72%); bare-metal: 13.8 KB (42%) y 1.0 KB (50%)	Se reemplazó el sketch por C++ bare-metal

Electrónica y alimentación

Prueba	Cómo	Resultado	Qué se cambió por ella
Choque de dirección 0x29	Se vio al tener un TCS34725 y un VL53L0X sin reasignar en el mismo bus (el mensaje del commit lo describe)	Las direcciones chocaban	ToF reasignados a 0x30+i antes de `init()` ; TCS al final
OLED no enciende al alimentar por VIN	Observación al prender por VIN (comentario del código)	La OLED tarda en despertar	`delay(500)` antes de iniciarla
Motores giran al revés	Prueba del robot (implícita en el cambio)	Sentido invertido	`INV_IZQ` e `INV_DER` en el sketch, en lugar de voltear cables
Cámara no aparece con el UNO Q alimentado por VIN	Manual de diagnóstico: medir al menos 4.8 V en el hub, `adb shell lsusb` , `adb shell ls /dev/video*` ; iteración de 4 commits sobre el arranque	Con VIN, el USB-C de la Q no da 5 V al hub; el hub necesita su puerto PD	Hilo `vigilar_cam` que busca y reconecta cada 2 s; manual de alimentación
Test de alimentación sin PD (`docs/uno_q.md` , sección 12)	Q por VIN, hub sin alimentar, cámara conectada; `lsusb` y `v4l2-ctl --list-devices`	Pendiente	Decidirá si se compra el módulo PD
Voltaje mínimo de batería	Criterio escrito en los manuales: VIN de al menos 7 V; operar con 7.4 V o más; con menos de 7 V el L298N pierde fuerza	Los commits no dicen cómo se midió	Aviso de recargar a 7.2 V y requisito de 7.4 V en el manual de uso
Compilación en el core Zephyr	Compilar el sketch en el UNO Q (fallos implícitos en los commits)	Adafruit_SSD1306 no compila	Driver OLED propio (`oled.h`)
Arranque de App Lab	Arranque con archivos de datos	Imágenes en `/app` y un JSON vacío hacían fallar el arranque	Datos en `/app/datos` con escritura atómica y lectura tolerante

Sensores

Prueba	Cómo	Resultado	Qué se cambió por ella
Frontal contra pared	Lectura real del ToF frontal (comentario del sketch)	El sensor “nunca llega a 50 mm”; con pared a menos de 130 mm hay pared y de 140 para arriba no	`D_FRENTE` de 55 a 130 mm, `PARED_F = 130`, `D_CERCA` de 15 a 10; frenado desde 230 mm
Lecturas por posición de pared	Lecturas observadas del frontal y de los laterales (notas de la sesión)	Frontal: pared en su celda unos 130 mm, una celda más allá unos 310 mm. Laterales: pared unos 50 mm, libre más de 200 mm	Umbral de frente libre a la mitad, 220 mm; umbral lateral de 120 mm
Colores de fábrica del TCS34725	Lecturas reales del sensor; el panel promedia 5 lecturas con 50 ms entre una y otra	PISO 0.282, 0.379, 0.333 con brillo 153; AMAR 0.409, 0.449, 0.142 con brillo 1836; ROJO, VERDE y AZUL en los commits	`REFS` y `refs[]` pasan de valores provisionales a valores de fábrica
Falsos positivos sobre el piso negro	Observados con el sensor de color (el comentario habla de quitar falsos positivos)	Lecturas de color falsas sobre piso negro	`BRILLO_MIN = 1.3` en Python y sketch; rechazo si la cámara vio piso limpio (luego la cámara se simplificó)
Cámara para paredes (etapas 20 a 25)	Se usó como sensor de paredes, luego como guía (7c1b4be) y se revirtió (8c37fe8); pruebas en pista del equipo	Veía demasiadas paredes y zapatos (ve muy por arriba); con mucho peso chocaba todo el tiempo; sus regiones dependían de la inclinación, que cambió de 25 a 40 grados	Se quitó de la estrategia; queda solo para amarillo y blanco/negro
Alinear con giros de 5 grados	Se implementó (8b497c4) y se quitó 15 minutos después (0c72165)	Prueba negativa: el robot se volvió súper lento (pruebas del equipo)	Avance derecho con centrado solo si un lateral lee menos de 90 mm

Software

Prueba	Cómo	Resultado	Qué se cambió por ella
Orden de salidas	Prueba automática en Python con datos simulados, sin robot	Pasó: frente, izquierda, derecha, media vuelta	Regla de prioridad de salida
Umbral de frente libre	Prueba automática con datos simulados	Pasó (220 mm)	Confirmó el umbral de 220 mm
Ruta más corta (BFS)	Prueba automática en un mapa de prueba	Pasó	Ruta de la segunda corrida
Votos del mapa	Una lectura mala no borra una pared vista 3 veces	Pasó	Mapa con votos y tope ±5
Decisión tras un salto	Prueba automática: la salida nueva más lejana	Pasó	Salida tras cambio brusco de ToF
Guardar y cargar el mapa	Prueba automática con datos simulados	Pasó	Escritura atómica del mapa
Compilación del Python	`py_compile` en cada cambio	Sin errores de sintaxis	Ninguno
Sin cámara	`USAR_CAM = False`	Permite aislar ToF y sensor de color	Se usó para depurar la estrategia de seguir pared

Herramientas de prueba integradas en el robot

Herramienta	Cómo se usa	Para qué sirve
Modo calibración	Botón al encender; muestra r, g, b, c, el color y las 3 distancias en la OLED	Calibrar colores y ver los ToF sin PC; en la iteración A, el botón corto avanza una celda y el largo gira 90 grados
Panel web de colores con asistente	Puerto 7000; promedia 5 lecturas y guarda en `colores.json`	Calibrar colores desde el teléfono sin tocar `refs[]`
Simulador “qué haría” (`que_haria()`)	Panel “Ver sensores ToF”; no mueve el robot	Probar la decisión con lecturas ToF vivas sin moverlo
Vista de cámara con zonas	`/api/foto` a 160×120; recuadros de amarillo y de zona de paso	Ajustar `ZONA_PASO` hasta que en un pasillo libre salga verde
Contador `PY n` y fase en la OLED	OLED en reposo; la fase (SENSA, DECIDE, GIRA, AVANZA, ESCANEA) también en la web	Ver si Python corre y en qué estado está
`USAR_CAM` y `odo(8)`	Constantes y RPC	Aislar fallos de la cámara y leer el desvío en grados

Pendiente

Prueba	Cómo	Resultado	Qué se cambió por ella
Pendiente: probar en pista la última versión	Correr la pista con distancia, saltos, votos, máquina de estados y MPU	Aún no; hasta ahora solo hay pruebas automáticas con datos simulados	Por definir con los datos de la pista

Decisiones de ingeniería

EN CORTO

Dieciocho decisiones, cada una con sus alternativas, el criterio y lo que se eligió. La idea que se repite: los ToF mandan, y una lectura mala nunca decide sola.

Para cada decisión se indican las alternativas que aparecen en el historial, el criterio y lo que se eligió. Cuando una alternativa no tiene respaldo en un commit o en las notas, se dice.

1. Movimiento por tiempo, luego por distancia.
Alternativas: encoders (primera versión, `TICKS_CELDA` 360), tiempo (`MS_CELDA` 600) y distancia con el ToF frontal. Criterio: los N20 no tienen encoder; el tiempo depende de batería, piso y carga, y el robot a veces se pasaba de la celda o no encontraba nada; el ToF mide directamente lo que falta. Decisión: por distancia (parar cuando la lectura baja 180 mm), con tiempo solo como plan B cuando el frontal no ve pared a menos de 1 m y como tope (`t > 3 * dur`). Python vuelve a leer los 3 ToF (mínimo de 3 lecturas) antes de decidir.

2. Cámara para paredes o ToF.
Alternativas: cámara con regiones (hasta 2 celdas adelante), cámara que encarece, ToF solamente. Criterio: en pista la cámara veía demasiadas paredes (ve muy por arriba: paredes lejanas, zapatos) y el robot no sabía a cuál darle prioridad; sus regiones se ajustaban a mano y dependían de la inclinación, que cambió de 25 a 40 grados; si se equivoca, un reflejo o mala luz podría encerrar al robot. Mejora futura: calibrar la exposición y los parámetros de la cámara para distinguir mejor blanco y negro. Decisión: los ToF mandan para paredes; la cámara se queda en amarillo y en distinguir blanco (pared) de negro (piso), y nunca bloquea un camino. El frontal hace el doble check y el sketch para a 130 mm. Se acepta perder la anticipación a cambio de simplicidad y robustez.

3. Ultrasónicos a 45 grados o VL53L0X a 45 grados para las esquinas ciegas.
Alternativas: HC-SR04 a 45 grados o 2 VL53L0X más a 45 grados. Criterio: a 45 grados el sonido rebota de lado en una pared lisa (reflexión especular) y casi no regresa eco; el cono de unos 30 grados del HC-SR04 a 150 mm ilumina una mancha de unos 80 mm (lecturas ambiguas); el ECHO a 5 V requiere divisor para el STM32 de 3.3 V; las versiones I2C de 5 V subirían SDA/SCL a 5 V en un bus de 3.3 V; medir con `pulseIn` bloquea el control de 10 ms y hay que disparar alternado por crosstalk. Decisión: ultrasónicos descartados por ahora. Si hace falta, 2 VL53L0X más a 45 grados (mismo bus I2C, 2 pines XSHUT, cono de unos 25 grados, mismo soporte impreso). No se implementó.

4. Umbral de frente libre a la mitad (220 mm).
Alternativas: 150 mm (se había usado) o 220 mm. Criterio: pared en la celda propia da unos 130 mm y una celda más allá unos 310 mm; con 150 mm quedaban solo 20 mm de margen frente al ruido de los ToF (unos ±15 mm medidos en pista). Con 220 mm el margen es de unos 90 mm (frontal: 130 contra 310) y en el lateral de unos 70 mm (120 contra 50 y más de 200), muy por encima de ±15 mm. Decisión: 220 mm, a mitad de camino entre ambas lecturas. Además, cada ToF se puede calibrar con un offset midiendo a una distancia conocida. El mismo criterio dio el umbral lateral de 120 mm (pared unos 50 mm, libre más de 200 mm).

5. Votos en el mapa.
Alternativas: tomar la última lectura o acumular evidencia. Criterio: una sola lectura mala sobrescribía una pared bien vista. Decisión: cada lectura suma +1 (pared) o -1 (libre), con tope ±5 (`VOTO_MAX`) para que el mapa se pueda corregir; por donde pasó el robot queda libre seguro.

6. Máquina de estados explícita.
Alternativa: decidir mientras se mueve. Criterio: decidir con lecturas frescas y el robot quieto, más una fase visible para depurar. Decisión: SENSA, DECIDE, GIRA, AVANZA (ESCANEA tras un salto) en Python; el STM32 conserva su máquina de tiempo real (NADA/AVANZA/GIRA) y el color se revisa todo el tiempo.

7. Carga de la página web.
Alternativas: que la web consulte al Bridge siempre o que lea valores guardados durante la corrida. Criterio: los motores no se afectan, pero cada consulta eran 5 a 9 llamadas al Bridge cada 150 a 300 ms que compiten con el robot, y la foto bloqueaba la cámara unos 130 ms; el riesgo era un “alto” tardío en el amarillo. Decisión: durante la corrida la web lee los últimos valores y la última foto, sin Bridge; el STM32 para solo al confirmar amarillo; la OLED no se refresca en movimiento (unos 25 ms bloqueantes).

8. Reglas de decisión.
Alternativas: mano derecha, mano izquierda, mapa con flood-fill (BFS, luego Dijkstra), contar visitas. Criterio: se probó mano derecha y luego izquierda. Según las pruebas en pista, la mano derecha llegó más lejos y se movía más, y con la mano izquierda se quedaba atorado en una misma zona. La regla “frente primero, luego izquierda” ya no es un seguidor de pared puro: un seguidor de pared solo garantiza recorrer todo el muro si siempre prefiere el lado de su mano; ir de frente primero puede dejarlo dando vueltas en una zona, lo que coincide con lo observado. Las islas se rompen con 3 visitas (`ISLA`) tomando la salida menos usada. Tras un cambio brusco se va a la salida nueva, no mapeada, de lectura más lejana. Decisión: volver a la mano derecha pura (derecha, frente, izquierda, media vuelta); el código actual aún tiene frente, izquierda, derecha y está pendiente de cambiar. BFS sobre el mapa solo para la ruta corta de la segunda corrida.

9. Cambios bruscos de ToF.
Alternativa: avanzar la celda completa y leer al inicio. Criterio: se pasaban por alto aberturas laterales a medio avance; la lectura que salta de 50 a 200-300 mm (o de 100 a 300) indica que algo se abrió, pero una sola lectura mala no debe disparar nada. Decisión: el salto cuenta si se repite en 2 lecturas (`SALTO = 120` mm); si se abre un lateral, el robot avanza `TRAS_SALTO = 124` mm (90 + 40 - 6, del CAD) hasta el centro de la celda nueva y escanea. Eso además re-ancla la odometría.

10. MPU-6050 opcional y autodetectado.
Alternativas: giros por tiempo (`MS_90`), alinear con barridos de 5 grados, giroscopio. Criterio: los giros por tiempo son la mayor fuente de error (un giro de 80 grados tuerce todo lo que sigue); el barrido de 5 grados fue lento y a lazo abierto; el UNO Q no trae IMU y puede que no se consiga a tiempo. Decisión: un solo sketch que, si el GY-521 responde en 0x68, gira por ángulo absoluto y sostiene el rumbo, y si no, sigue por tiempo. El signo del montaje se aprende en el primer giro y el sesgo se mide al encender (200 muestras).

11. C++ bare-metal en el ATmega por memoria.
Alternativa: el sketch Arduino (18.9 KB de flash al 58% y RAM al 72%). Criterio: la RAM al 72% dejaba poco margen y la materia pide Microchip Studio. Decisión: C++ con registros (13.8 KB de flash al 42% y 1.0 KB de RAM al 50%), con TWI a 400 kHz, Timer1 CTC a 1 kHz y Timer0 Fast PWM.

12. Bus I2C con 0x29 compartido.
Alternativas: un segundo bus o hardware extra, o reasignar direcciones. Criterio: el TCS34725 y los VL53L0X responden en 0x29. Decisión: reasignar los ToF a 0x30+i con sus pines XSHUT antes de `init()` y arrancar el TCS al final, sin hardware adicional.

13. Chasis modular R3 / UNO Q.
Alternativas: dos chasis, o uno con variantes. Criterio: el R3 es respaldo oficial y la Q es la placa experimental pero principal; el R3 es de 5 V y la Q de 3.3 V (a 5 V se dañan los pines I2C de la Q). Decisión: un solo chasis con `PLACA = nano | unoq | uno_r3` , mismo mapa de pines, un shield con dupont que se pasa de una placa a otra y un jumper de VCC 5V/3V3.

14. Driver OLED propio por Zephyr.
Alternativa: Adafruit_SSD1306. Criterio: no compila en el core Zephyr del UNO Q. Decisión: `oled.h` propio, sin librerías.

15. Cámara USB UVC en lugar de MIPI-CSI.
Criterio: la CSI exige la Media Carrier, más grande y de 1.8 V. Se mantiene el TCS34725 porque la cámara anticipa pero no da una lectura de color exacta; en pista es el mejor componente. Alimentación: el UNO Q no siempre reconocía la cámara aunque tuviera 5 V; se resolvió con un adaptador OTG "tonto" de USB-C a USB-A, y el Arduino va vertical con cinchos para que el OTG no salga del contorno.

16. Soporte del ToF como pieza aparte con insertos.
Alternativas: integrado en el piso 1, postes con ranura de 1.8 mm, bloque detrás del módulo con insertos. Criterio: reimprimir una pieza pequeña y no el piso; poder roscar en los laterales donde la columna tapa la cara trasera; dejar el header y los dupont libres. Decisión: bloque con 2 insertos y M3×10 por el frente, con 2.2 mm de pared.

17. Colores: referencias de fábrica, brillo y tamaño de zona.
Criterio: en pista, entre más grande la zona de color, mejor, porque el sensor RGB puede pasar por un lado y saltarse una línea amarilla delgada; por eso las marcas deben ser anchas. Además: normalizar r, g, b entre sí hace la clasificación menos sensible a la luz; el brillo c, con peso pequeño, distingue el piso de las marcas. Decisión: referencias calibradas guardadas en `colores.json` y recargadas al arrancar, con tres lecturas seguidas para aceptar un color.

18. Alto en amarillo con doble confirmación.
Criterio: evitar parar por un falso positivo de un solo sensor, pero sin esperar a Python. Decisión: el STM32 llama `termina()` al confirmar el amarillo con el TCS34725; la cámara ayuda a buscar el amarillo pero no es requisito de la parada.

Arquitectura según el grafo de conocimiento

EN CORTO

El grafo de graphify confirma que los VL53L0X son el centro del robot y que todo en el UNO Q pasa por el Bridge. Señala `main.cpp` del ATmega como el archivo a dividir.

El repositorio se analizó con graphify, que arma un grafo de conocimiento con el código, los documentos y el CAD. Cada función, clase o sección es un nodo y cada llamada o referencia es una relación. El reporte completo está en `graphify-out/GRAPH_REPORT.md` y el grafo interactivo en `graphify-out/graph.html`. Se generó el 11-nov desde el commit `c3ab381`, antes de la última etapa. Para actualizarlo: `graphify update .`.

Dato	Valor
Archivos analizados	42 (~47,600 palabras)
Nodos / relaciones	562 / 1011
Comunidades (grupos de piezas muy relacionadas)	27
Relaciones extraídas directo del código / inferidas	88 % / 12 %
Ciclos de dependencias entre archivos	Ninguno

Nodos más conectados

Nodo	Relaciones	Qué significa
`regAddr` (registros del VL53L0X)	59	El driver propio del sensor de distancia es la pieza más grande del firmware del ATmega.
`VL53L0X`	46	Es el mayor puente entre comunidades (centralidad 0.254): une firmware, guía de programación y la versión UNO Q. El grafo confirma que los sensores de distancia son la columna del robot, y por eso “los ToF mandan”.
`Oled`	21	Driver propio de la OLED; se escribió porque la librería de Adafruit no compila en Zephyr (UNO Q).
`rpc()`	19	En el UNO Q todo pasa por el Bridge entre Linux y el STM32. Por eso se cuidó la carga de la web.
`calibracion()`	18	El modo calibración (botón al encender) toca casi todos los sensores; es la herramienta de prueba central.
`avanzar()`	12	El avance es el punto donde se juntan sensores, cámara y mapa.

Lo que el grafo señala para mejorar

- `main.cpp` del ATmega tiene cohesión de 0.07: muchas funciones en un solo archivo y poco relacionadas entre sí. Conviene dividirlo en módulos (movimiento, sensores, mapa).

- Hay 87 nodos aislados. Casi todos son registros del VL53L0X que se definen y no se usan; no es un hueco de diseño.
- La documentación está enlazada al código: el manual técnico, la guía y el README de cada iteración aparecen conectados con las funciones que describen.

Lecciones aprendidas

EN CORTO

Lo que el equipo se lleva: calibrar desde la web, umbrales a la mitad de lecturas reales, una jerarquía clara entre sensores y que quitar código también es ingeniería.

- **Calibrar desde la página web fue de lo más útil del proyecto.** Colores, distancia, simulador de decisiones y cámara se ajustan desde el celular en la pista, sin reprogramar ni copiar valores; por eso cada nueva calibración se agregó ahí.
- Un número geométrico no sirve si el sensor no lo alcanza. `D_FRENTE = 55` nunca se cumplía porque el VL53L0X "nunca llega a 50"; con 130 mm, medido, el robot dejó de empujar la pared.
- Los umbrales se ponen a la mitad de las dos lecturas reales, no pegados a una. El frente a 150 mm dejaba 20 mm de margen entre 130 y 310; a 220 mm el margen es de unos 90 mm por cada lado.
- Cada fuente de información debe tener un papel y una jerarquía escritos. La cámara pasó de mandar a encarecer, a desempatar y a buscar amarillo en varios commits; el resto se resolvió cuando los ToF quedaron como autoridad para paredes.
- Una lectura mala no debe cambiar nada sola. Tres defensas lo muestran: mínimo de 3 lecturas, confirmación del salto en 2 lecturas y votos con tope en el mapa.
- Medir las piezas reales antes de modelar ahorra impresiones. La rueda real (45 × 17), el módulo VL53L0X (11 mm) y el motor (12 × 11) corrigieron el CAD, y el hueco del motor pasó por cinco valores entre 11.85 y 12.05 mm.
- El tiempo no es una unidad de distancia: el avance por tiempo deriva con batería, piso y carga, y los giros por tiempo acumulan error de rumbo. La referencia debe venir de un sensor.
- La carga lateral importa: la web que consulta al Bridge, la OLED que bloquea 25 ms y los archivos en `/app` pueden causar fallos que no son del algoritmo.
- Probar en pista temprano evitó seguir afinando ideas que no servían: el avance y el giro por tiempo fallaban por la batería, y alinear con giros de 5 grados volvía al robot súper lento.
- Un seguidor de pared solo es completo si siempre prefiere el lado de su mano; mezclar "frente primero" lo dejó atorado en una zona.
- Una zona de color delgada se puede saltar: el sensor RGB pasa por un lado, así que las marcas deben ser grandes. Y un componente "tonto" (el adaptador OTG) puede resolver un problema de alimentación que parecía de diseño.
- Quitar código también es una decisión de ingeniería. `alinear()` se retiró tras 15 minutos y la reescritura quitó unos 500 renglones de `main.py` sin perder capacidad, porque los ToF ya hacían ese trabajo.

Pendientes

EN CORTO

Lo que falta antes de dar el robot por terminado: probar en pista la última versión y calibrar sensores, cámara y MPU con datos reales.

- Probar en pista la última versión: avance por distancia, cambios bruscos, mapa con votos, máquina de estados y MPU-6050.
- Verificar que las paredes de madera estén pintadas de blanco, porque la detección de la cámara toma blanco como pared.
- Calibrar los 3 ToF en la pista desde la web (un punto a 100 mm por sensor: solo desfase; el equipo lo simplificó así en pista) y, con esas lecturas, revisar `PARED`, `PARED_F`, `D_FRENTE`, `D_LADO_MIN` y `FRENTE_LIBRE`. Va **antes** de calibrar la distancia por cámara, que usa el frontal como regla.
- Calibrar la cámara en la pista desde la web (exposición, derecho, escala) y medir su placa para el CAD.
- Probar la corrección de rumbo con la cámara real (con y sin MPU) y, después, la versión experimental (distancia por cámara y aberturas).
- Conseguir el MPU-6050 (GY-521) y calibrar `ADELANTO` (si se pasa, subirlo; si le falta, bajarlo) y `KG`.
- Calibrar `ZONA_PASO` y `BLANCO_MAX` hasta que en un pasillo libre el recuadro de la vista de cámara salga verde.
- Calibrar `TRAS_SALTO` (124 mm, calculado del CAD) y `BORDE_AJ`, con el robot en la pista.
- Usar los colores leídos para corregir la posición en el mapa (hoy se anotan y se cuentan, pero no corrigen).
- Dibujar el mapa en la web.
- Hacer el test de alimentación sin PD (`docs/uno_q.md`, sección 12) y decidir si hace falta el módulo PD.

Glosario

EN CORTO

Términos que se repiten en la bitácora. El glosario completo está en el Manual técnico.

Término	Significado
Bare-metal	Firmware que programa los registros del microcontrolador directamente, sin el core de Arduino
BFS	Búsqueda en anchura: encuentra el camino más corto entre dos celdas
Bridge	Servicio que pasa las llamadas (RPC) entre Linux y el STM32 del UNO Q
CAD	Modelo 3D del chasis (OpenSCAD) del que salen las medidas del robot
Commit	Cambio guardado en el repositorio de git; cada etapa cita los suyos
(deducido)	Motivo o prueba que no está escrito en el commit, el código ni en las respuestas del equipo
DFS	Búsqueda en profundidad: avanza hasta el fondo de un camino antes de volver a probar otro
Flood fill	Algoritmo que llena el mapa con la distancia a la meta; se usó en la primera versión
MPU-6050	Sensor de giro y aceleración (GY-521)
Otsu	Método que elige solo el umbral que mejor separa piso negro y pared blanca
RPC	Llamada a procedimiento remoto: un procesador le pide a otro que ejecute una función
ToF	*Time of Flight*: sensor láser de distancia (VL53L0X)
Trémaux	Método clásico para recorrer un laberinto marcando los caminos ya visitados
Voto	Cada lectura suma +1 (pared) o −1 (libre) a un lado de la celda, con tope ±5